

Using Local Coding Agents

Using Open-Weight Models in Local Coding Harnesses as an Alternative to Claude Code and Codex Subscriptions



SEBASTIAN RASCHKA, PHD

JUN 27, 2026

♡ 202

💬 20

🔄 24

S

Many people reached out to me in the past asking about my local agent stack as well as I set up my local agent stack.

So, I thought it might be useful to put together a little tutorial on how to set up a local (co) agent using open-source tools and open-weight LLMs.

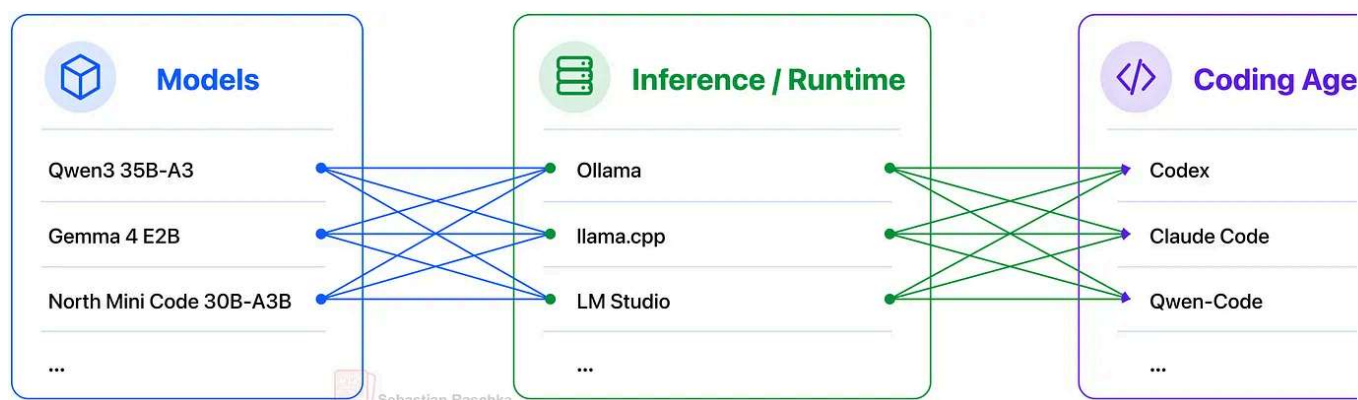


Figure 1: Overview of the local stack, that is, a coding agent harness that uses a local model hosted through an inference engine / runtime server.

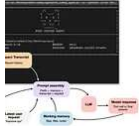
This article is a tutorial on setting up a production-ready coding agent with a fully local stack. We will use a locally served LLM together with a local coding harness that can read files, make edits, run commands, and verify changes as shown in the figure above.

Here, we can think of the LLM as the engine that provides the reasoning and code generation. And the surrounding harness provides the operating environment that allows the LLM to do meaningful coding work in our local projects.

Why local? For many coding workflows, a local setup is an interesting alternative to proprietary services such as GPT in Codex or Opus in Claude Code. The local setup is

transparent, inspectable, and free to run apart from hardware and electricity costs. It also stays fully under your control, and you can modify the coding harness in any way you like **Plus, it's a lot of fun!**

By the way, in case you want a bit more background information on coding agent harnesses covered the core components of coding agents (and building a coding agent from scratch for learning purposes) here:



Components of A Coding Agent

SEBASTIAN RASCHKA, PHD · 4月4日

Read full story →

1. Intro

I have to admit that I still primarily alternate between Codex and Claude Code as my daily drivers, for now (and just to keep up with the new tooling and functions that are constantly being added). Also, the plan limits (especially for Codex) are still so generous that I haven't had to worry about costs so far.

However, I've been using local solutions for a while, too, to test things and because it somehow gives me joy to have and use a fully local setup (versus proprietary services).

Either way, local solutions become more and more attractive each day. One aspect is the costs. If you have the hardware, they are practically free to run. And then there's, of course, the privacy angle. For example, for organizing and processing my receipts, I'd be more comfortable with a local model ingesting them rather than sending the data over to OpenAI or Anthropic.

(Then, if we keep in mind that Anthropic was recently [throttling their flagship model's performance for LLM research](#), proprietary services may become more restrictive over time and it's maybe a good idea to be comfortable with open-weight alternatives as a backup.

And there are many, many additional reasons and use cases like that.

Your motivations for using local LLMs and coding harnesses may include:

- Predictable, fixed costs if you reach your subscription plan limits, and immunity to AI price changes.
- Reproducibility; sometimes it's nice if a model is upgraded (e.g., GPT 5.4 -> GPT 5.5! GPT 5.6) and it solves all your queries more reliably. However, this can also break existing workflows.
- Offline use in the classic airplane flight scenario with slow or no internet, or when going on a coding/writing retreat in the cabin in the woods w/o a Starlink subscription.

And there are probably several others.

So, in this article, we will set up and use popular harnesses like Codex and Claude Code open-weight models and investigate whether using a model-specific harness (like Qwen Code for Qwen3.6) brings any additional benefits. (Of course, there are many more harnesses like OpenCode, Cline, Pi, and Noumena Code, but I thought that most people already have muscle memory with either Codex or Claude Code, which makes switching open-weight models a bit smoother).

2. Coding Agent Harness Overview

Most coding agent harnesses follow similar principles and have more or less the same features and functionality. However, the implementation details may differ, and certain LLMs have usually been primarily optimized for a specific harness. Of course, many open-weight LLMs like GLM 5.2, for example, would run Claude Code, etc.

However, if an LLM developer also develops a coding harness, it is somewhat safe to assume that their model is optimized for their own harness first (while also supporting others).

Here, I am primarily going to use Qwen3.6 with the Qwen-Coder coding client. However, I will also go over other options for using a local LLM with other agent harnesses, for example Claude Code, Codex, and the increasingly popular Cline, but more on that later.

The reason why I am primarily using Qwen-Code when working with Qwen models is that

1. it is open-source, like Codex (<https://github.com/openai/codex>) but unlike Claude C

2. Qwen models have been specifically optimized for the Qwen-Code harness (more information below);
3. I can run both Codex (with the latest GPT model) and Qwen-Code with a local Qwen model side by side on the same machine without having to switch manually back and forth between models.

Regarding the second point in the list above, that Qwen models work better in Qwen-Code, Nvidia's [Polar: Agentic RL on Any Harness at Scale](#) paper (May 2026) has a benchmark showing that the Qwen3.5-4B base model has the best coding performance in said Qwen-Code harness (both before and after their Polar-RL training), which I included below.

Qwen3.5 performs much better than in open-source Codex and slightly better than in the closed-source Claude Code

Harness	Base	POLAR RL	Gain
Codex	3.8%	26.4%	22.6 ↑
Claude Code	29.8%	34.6%	4.8 ↑
Qwen Code	34.6%	35.2%	0.6 ↑
Pi	34.2%	40.4%	6.2 ↑

SWE-Bench Verified scores for the Qwen3.5-4B

Figure 2: Qwen model performance in different coding harnesses via *Polar: Agentic RL on Any Harness at Scale* (<https://arxiv.org/abs/2605.24220>)

The benchmark in the table above is for an older Qwen3.5 model, and I am assuming that the latest Qwen3.6 models are even further optimized to do well in Qwen-Code specifically.

However, Pi (<https://github.com/earendil-works/pi>) also seems to be a very interesting candidate that I need to play around with in the future.

By the way, Qwen3.6 35B-A3B is about 22 GB to download, requires roughly 30-40 GB RAM, and runs pretty swiftly on both a Mac Mini with M4 and a DGX Spark.

Based on the recent benchmarks shared by Cohere earlier in June, it is currently the best local model in its size class.

Agentic coding tasks

Best model per category highlighted in red

Benchmark	North-Mini-Code (30B-A3B)	Qwen3.6 (35B-A3B)	Poolside XS.2 (33B-A3B)	Gemma4 (26B-A4B)	Devstral Sm (24B Dense)
Terminal-Bench v2	36.0	51.5	35.7	34.2	22.5
Terminal-Bench Hard	31.1	35.0	25.0 *	25.0	17.0
SWE-Bench Verified	67.6	73.4	69.9	17.4	68.2
SWE-Bench Pro	40.2	49.5	46.3	13.0	35.7
SciCode	38.2	35.8	31.9 *	40.0	28.8
LiveCodeBench v6	70.3	80.4	55.2 *	77.1	40.3 *

* Internally measured r

Single-prompt-based coding tasks

Figure 3: Cohere benchmark from North Mini Code report published in June
<https://huggingface.co/blog/CohereLabs/introducing-north-mini-code>

As seen above, Qwen3.6 35B-A3B dominates all but one benchmark in this size class. However, that being said, Qwen Code is a general harness and also supports other type models. For instance, we could also connect North Mini Code or Gemma 4 in Qwen Coc

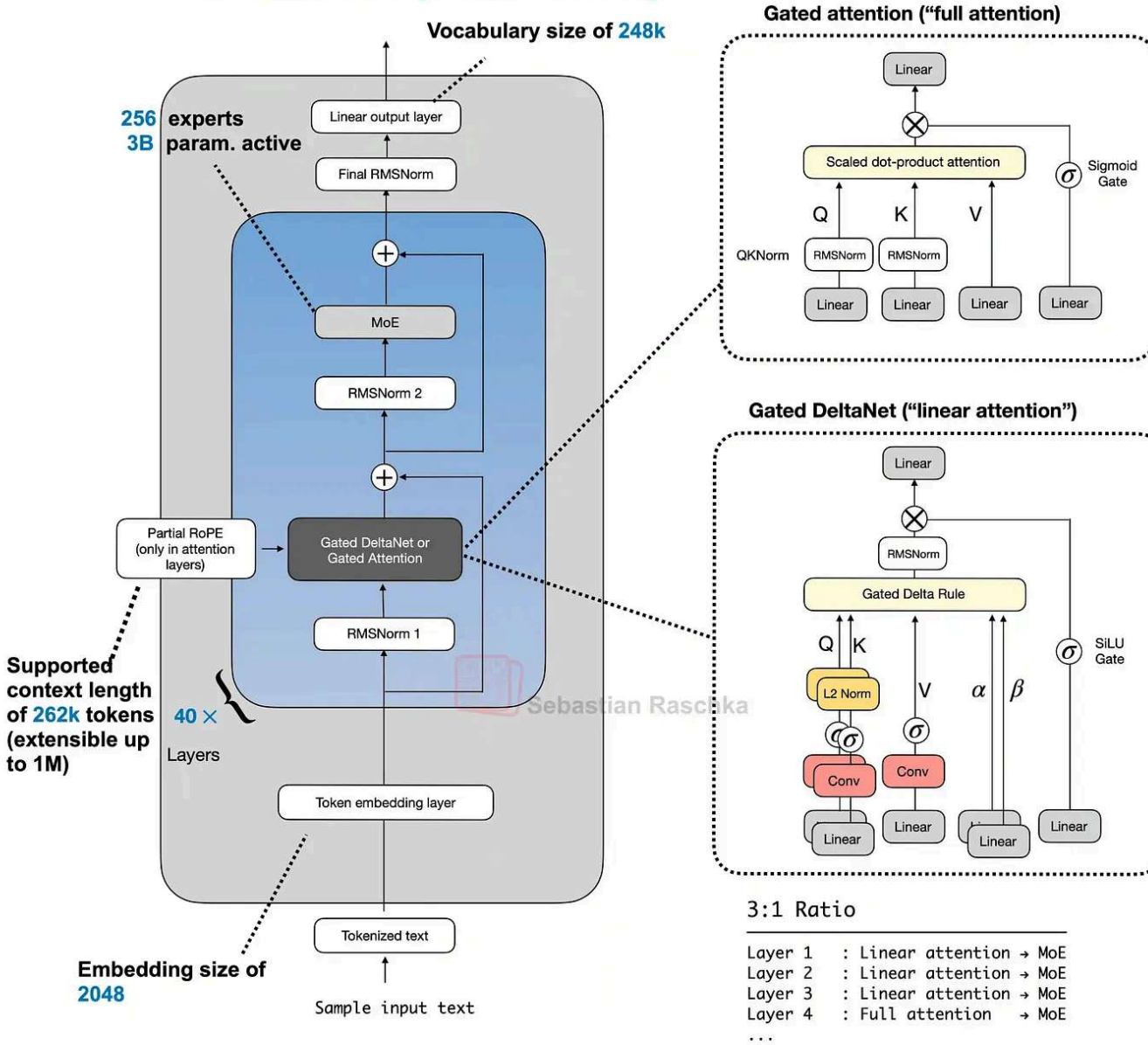
Model	Active Params	SWE-bench Verified
Claude Opus 4.6	~200B+	75.0%
Qwen3.6-35B-A3B	3B	73.4%

Same benchmark. 60x fewer active parameters.

Figure 4: Yes, Qwen3.6 35B-A3B is a really good model! (Via
x.com/pupposandro/status/2064707907489272147/)

Architecture-wise, the Qwen3.6 35B-A3B model has hybrid attention similar to Qwen3.5. I wrote more about it in [Beyond Standard LLMs](#).

Qwen3.6 (35B-A3B)

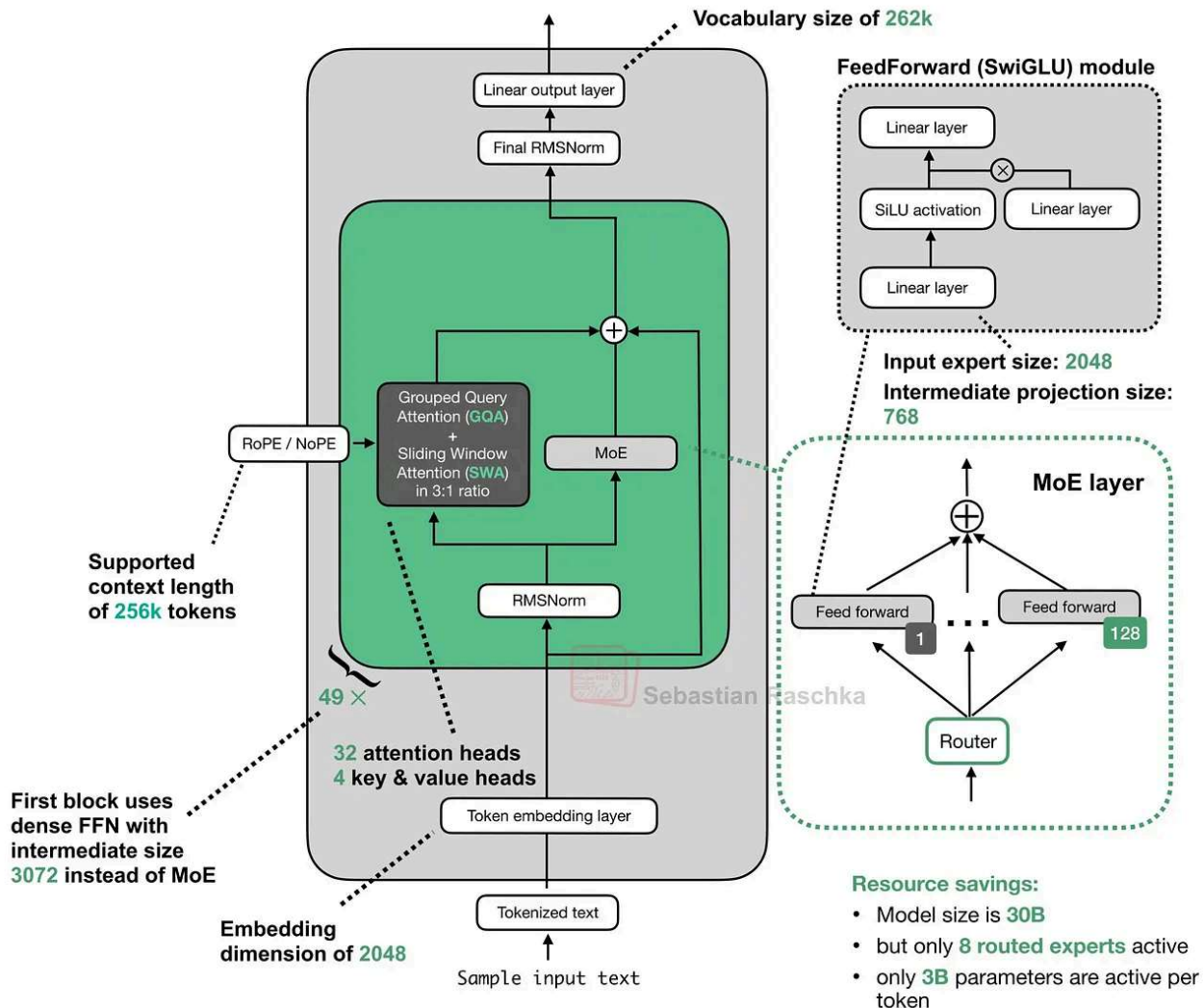


SCALE 35B total, 3B active (8.6% active)	CONTEXT (TOKENS) 262,144	DECODER TYPE Sparse hybrid	ATTENTION 3:1 Gated DeltaN and Gated Attenti
LICENSE Apache License 2.0	DATE 2026-04-15	LAYER MIX 10 gated attention + 30 DeltaNet	KV CACHE / TOKEN (BF16) info 20 KiB · Very low
KEY DETAIL Uses 256 experts with 8 routed plus 1 shared expert active inside a 40-layer hybrid stack.			

Figure 5: Qwen3.6 architecture and fact sheet from my LLM gallery.

Alternatively, if you don't want to use Qwen3.6, Cohere's North Mini Code is probably the most interesting, capable alternative at this size class right now. I will go over this model next local LLM setup section as well.

North Mini Code (30B-A3B)



SCALE

30B total, 3B active
(10% active)

CONTEXT (TOKENS)

256K input, 64K
output

DATE

2026-06-05

DECODER TYPE

Sparse MoE

VOCABULARY

262,144

LICENSE

Apache License 2.0

ATTENTION

8:1 GQA with 3:1
sliding-window/global
attention

LAYER MIX

36 sliding-window
13 global GQA/M

KEY DETAIL

Runs attention and MoE in parallel, with RoPE in sliding-window layers, NoPE in global layers, and top-8 routing over 128 experts.

KV CACHE / TOKEN

(BF16) [info](#)

98 KiB · Moderate

Figure 6: North Mini Code architecture and fact sheet from my [LLM gallery](#).

3. Local LLM Setup

No matter what agent harness we use (Qwen-Code, Codex, or Claude Code), we have to set up a local LLM, such as Qwen3.6 35B-A3B, first.

There are several options like Ollama, LM Studio, vLLM, SGLang, MLX, etc to serve models locally. You know from my Build A Large Language Model (From Scratch) and Build A Reasoning Model (From Scratch) projects that I like to code these myself. Implementing a model from scratch has the benefits that we understand the whole stack, plus we can monitor and further train and fine-tune it.

However, here, we just look for a model serving framework that has been super optimized for inference speed and resource needs since we don't plan to do any training or fine-tuning at this point. (We could, as an extra step, convert and import our own from-scratch fine-tuned model into these efficient serving stacks, but this is out of the scope for this article.)

For this tutorial, we will use [Ollama](#) as our efficient model serving engine because it's relatively easy to install and use from the command line across different operating systems (although LM Studio also added a non-GUI `llmster` client, but I am less familiar with it).

By the way, I am not affiliated with any of the tools mentioned in this article, but one nice thing about Ollama is that they also optionally support open-weight models hosted in the cloud, including the currently strongest open-weight model, GLM 5.2, which is too large to run locally on consumer hardware. (The cloud models are not free, of course, but have similar subscription plans as ChatGPT and Claude; it's still nice though that this option exists to conveniently test the latest state-of-the-art open-weight models "locally.")

Anyways, setting up Ollama is pretty straightforward, and you can find the official macOS/Linux/Windows download instructions on their [download](#) page.

After installing, I recommend downloading a model for a quick test run. For instance, on macOS, we can use the ollama app to download models directly via the GUI:

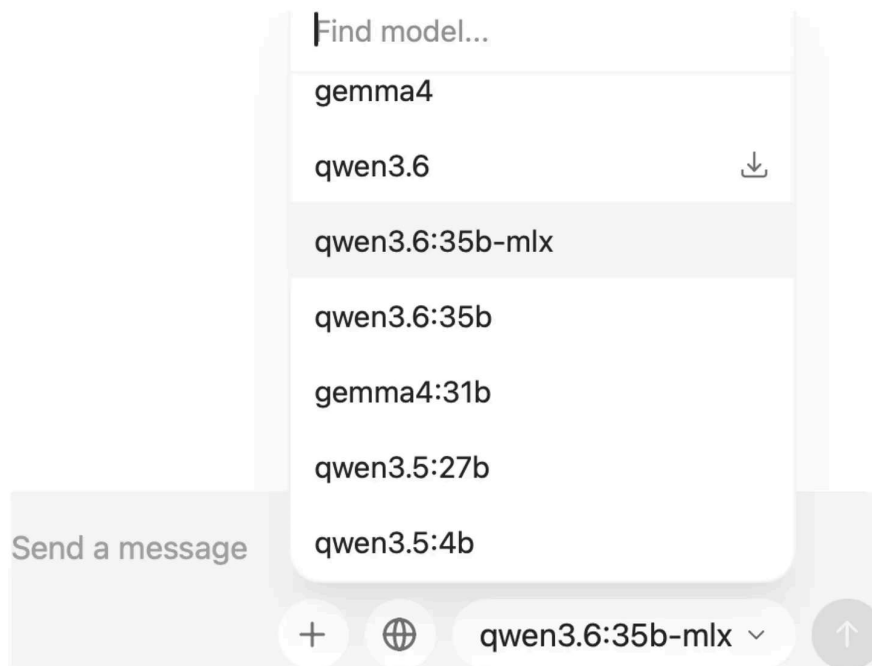


Figure 7: Using the Ollama app to find and download models

Otherwise, this can be done on the command line as well via

```
1 ollama pull qwen3.6:35b-mlx
```

By the way, the above-mentioned `qwen3.6:35b-mlx` is a model using Apple's Metal performance shaders, i.e., optimized for Macs with Apple silicon chips. I highly recommend using `*-mlx` versions of models working on Macs (if available).

ollama launch codex --model qwen3.6

OpenCode
ollama launch opencode --model qwen3.6

Models

More variants (and more efficient / quantized flavors) are usually available here [View all](#)

Name	Size / Usage	Context	Input
qwen3.6:latest	24GB	256K	Text, Image
qwen3.6:27b	17GB	256K	Text, Image
qwen3.6:35b latest	24GB	256K	Text, Image
qwen3.6:27b-mlx MLX	20GB	256K	Text, Image
qwen3.6:35b-mlx MLX	22GB	256K	Text, Image

On Macs (with Apple Silicon), use the mlx variant whenever you can

Figure 8: Prefer the MLX version when using a Mac (with an Apple Silicon chip).

On a Linux machine, use the non-MLX version:

```
1 ollama pull qwen3.6:35b
```

Then, to make sure that it works, you can either use the GUI again or launch Ollama from command line.

```

sebastian — rasbt@spark-e1bf: ~/Developer/local-agents-tutorial — ssh -t DGX-Spark cd /home/rasbt/; bas
[rasbt@spark-e1bf ~/Developer/local-agents-tutorial]$ ollama pull qwen3.6:35b
pulling manifest
pulling f5ee307a2982: 100% ██████████ 23 GB
pulling 5f3a3c817e78: 100% ██████████ 11 KB
pulling 86eff881e8d2: 100% ██████████ 94 B
pulling 5d1c86a949f7: 100% ██████████ 462 B
verifying sha256 digest
writing manifest
success
[rasbt@spark-e1bf ~/Developer/local-agents-tutorial]$ ollama run qwen3.6:35b
[>>> What is 1+1?
Thinking...
Here's a thinking process:

1. **Analyze User Input:** The user asks "What is 1+1?"
2. **Identify Core Question:** This is a basic arithmetic question.
3. **Calculate/Verify:** 1 + 1 = 2.
4. **Formulate Response:** Keep it direct and accurate. "1 + 1 = 2."
5. **Check for Edge Cases/Tricks:** None. It's a straightforward math question.
6. **Final Output Generation:** State the answer clearly.✅
...done thinking.

1 + 1 = 2

>>> Send a message (/? for help)

```

Figure 9: Running Ollama in the terminal.

You can exit this session via the `/bye` command.

As mentioned before, the currently best alternative to this Qwen3.6 35B-A3B model is the Mini Code 1.0 of similar size.

I suggest using this quantized version on Linux as it is a good trade-off between modeling performance and resource use (the “:latest” one is currently just a shorthand for “:q4_K_M”)

Search models

north-mini-code-1.0
North Mini Code is Cohere's first model for developers — a 30B Mi...

glm-5.2
GLM-5.2 is Z.ai's flagship model for the era of long-horizon tasks.

minimax-m3
MiniMax M3: Coding & Agentic Frontier. 1M context window. Native...

kimi-k2.7-code
Kimi K2.7 Code is Moonshot AI's coding-focused agentic model bui...

lrm2.5
LFM2.5-8B-A1B, an edge model built for fast, reliable tool calling o...

View all →

Models

Name	Size / Usage	Context	Input
north-mini-code-1.0:latest	19GB	488K	Text
north-mini-code-1.0:q4_K_M	19GB	488K	Text
north-mini-code-1.0:q8_0	32GB	488K	Text
north-mini-code-1.0:bf16	61GB	488K	Text
north-mini-code-1.0:mlx-mxvp8 (MLX)	31GB	488K	Text
north-mini-code-1.0:mlx-nvfp4 (MLX)	20GB	488K	Text
north-mini-code-1.0:mlx-bf16 (MLX)	61GB	488K	Text

I suggest using this quantized MLX version on macOS

Figure 10: North Mini Code 1.0 as an alternative to Qwen3.6 35B A3B.

4. Simple Speed Performance Assessment

Before deciding on whether to use an LLM as a local coding agent, it's usually not a bad idea to run a quick speed and quality assessment. Here, for the speed assessment, I would look for tokens/sec performance. Additionally, I'd also make sure this stays stable for (very) long contexts, which is what we are usually dealing with during agentic coding workflows (as opposed to simpler chatbots).

Of course, we also don't want the memory cost to explode either.

You could run my `ollama_speed_memory_bench.py` script to do a quick check. In a nutshell, it sends different prompts (ranging from 1k to 50k words) to an Ollama model and asks it to generate up to 8k tokens by default. It reports simple statistics like prefill speed from Ollama, prompt evaluation metrics, generation speed from output-token timing, and memory usage from the Ollama process plus NVIDIA GPU memory when available.

For example, to evaluate the `qwen3.6:35b-mlx` on macOS, if you downloaded or cloned the scripts from <https://github.com/rasbt/local-coding-agent-evals>, we can run the following command, which takes about 5 minutes:

```
1 uv run speed-memory-benchmark/ollama_speed_memory_bench.py --model qwen3.6:35b-mlx
```

On Linux, we can run:

```
1 uv run speed-memory-benchmark/ollama_speed_memory_bench.py --model qwen3.6:35b
```

Note that this assumes that you already downloaded the respective model as explained in the previous section. Also, depending on your system, if you have less than 30 GB RAM, you may have to use a smaller model like `gemma4:e2b`, which uses up to about 8 GB RAM on long contexts. Of course, there are also many smaller models, but in my experience, they make pretty bad local coding agents.)

Note that for models, the RSS RAM report is not super accurate on macOS (especially for model variants that utilize the Metal backend), and I suggest keeping an eye on the active

monitor's RAM usage for Ollama during the run as well. In this case, the RAM usage fluctuated between 20 - 29 GB.

Anyways, the bottom line is that for 50k contexts, the Qwen3.6 and North Mini Code model use up to 30 GB RAM and generate output with about 40 tok/sec on a recent Mac Mini and 30 tok/sec on a DGX.

Below is a visual summary of the different runs.

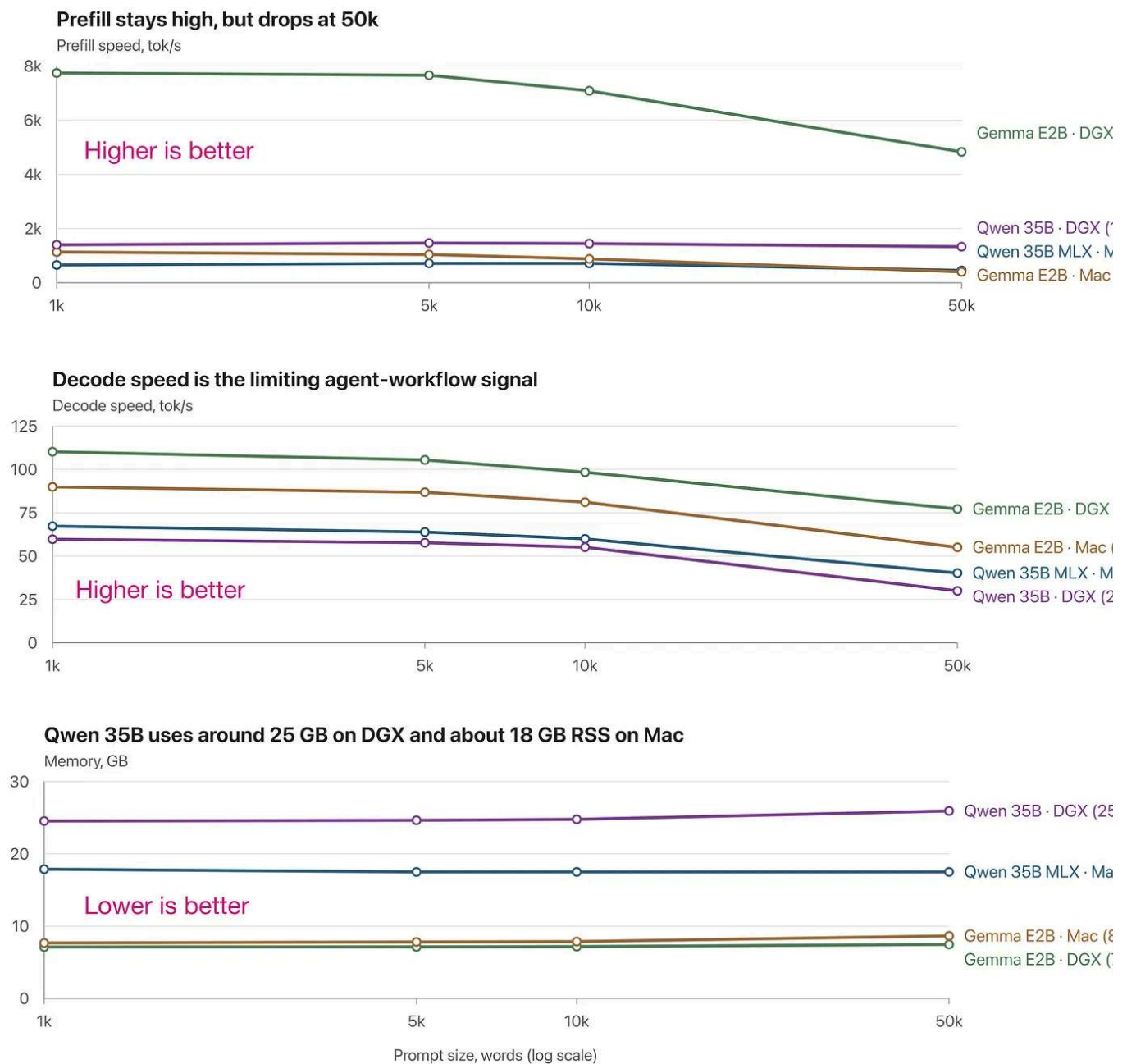


Figure 11: Quick speed comparison of the different models on different systems. Note that the macOS RAM consumption is not super accurate there. Also, note that the Qwen 35B-A3B model is faster on Mac than on the DGX Spark (which is the other way around for the Gemma 4 E2B model) thanks to the optimized MLX version. Code to reproduce:

<https://github.com/rasbt/local-coding-agent-evals>

Another interesting question is how Qwen 35B-A3B compares to the similarly-sized Co North Mini model? If we take similarly quantized models into account (above, I was using Qwen3.6 default), they are pretty similar, although North Mini is perhaps slightly ahead overall, as shown below.

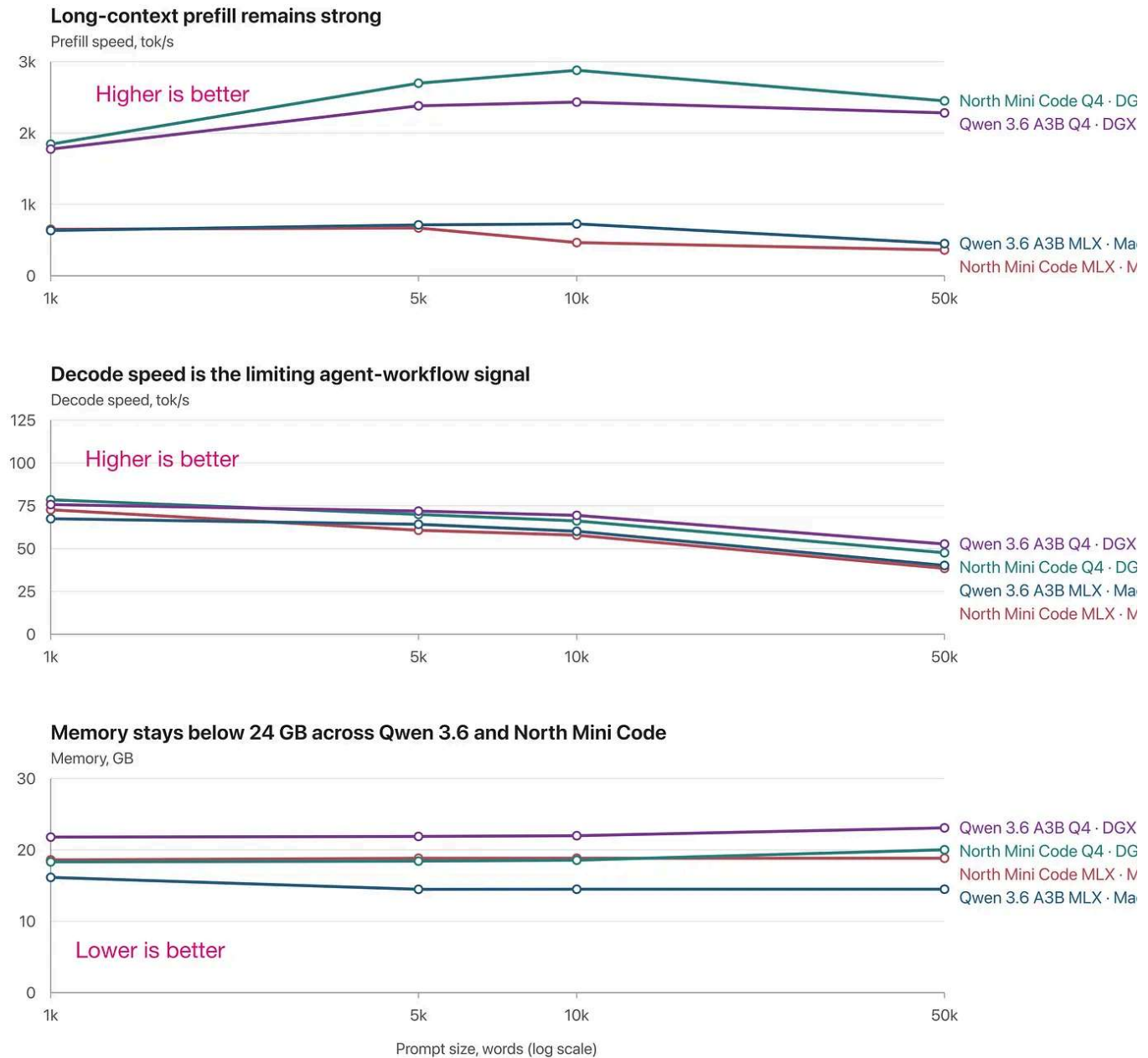


Figure 12: Q4-quantized Qwen3.6 35B vs North Mini Code. Code to reproduce:

<https://github.com/rasbt/local-coding-agent-evals>

Anyway, the bottom line is that, in my opinion, anything faster than 20–30 tok/sec is pret reasonable for local agent work. **This is about the same speed as GPT 5.5 with “high reasoning.** In this case, both models clear the bar easily.

By the way, personally, I run my agents almost exclusively on my DGX Spark because I don't want my Mac Mini to get too hot and I want to have the RAM available for other tasks.

Of course, there are always ways to optimize this more with different frameworks (other than Ollama), quantizations, MTP, and so on. However, Ollama is a good plug & play allrounder with minimal setup time that connects easily to various coding agent frameworks and with it's super simple to swap and try out different models.

5. Simple Benchmark Performance Assessment

After checking that the model is fast enough for convenient local work, I recommend doing a quick modeling performance assessment. Sure, there are many standardized benchmarks out there we could take a look at and even run ourselves.

Usually, you can find the numbers for relevant benchmarks in the model's technical report or model hub page. Usually, I also find it useful to look at a relative comparison with other models on <https://artificialanalysis.ai/models/>.

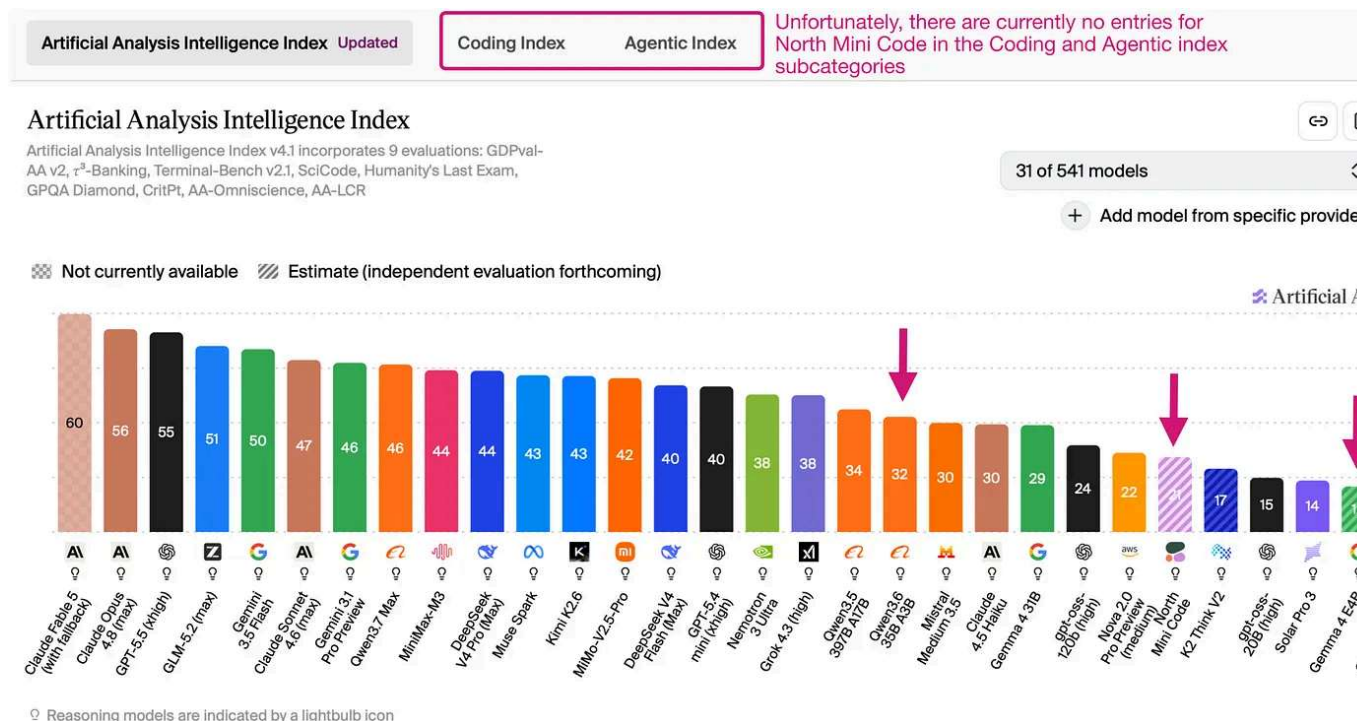


Figure 13: Benchmark from <https://artificialanalysis.ai/models/>. Average performance (top), coding performance (center), agentic performance (bottom).

Based on the figure above, we can see that Qwen3 35B-A3B is much more capable than Gemma 4 E4B and E2B models, for example.

Note that the Artificial Intelligence Index numbers keep changing over time as they swap benchmarks and update the weighting, so there are no “absolute” numbers we could use as a reference point for deciding which model is “good enough”. Rather, I would compare a new interesting model to a model you used before as an anchor or reference point.

Beyond standard benchmarks, I would also curate a personal set of tasks that are relevant to you to do a quick check whether this model is even suitable for any type of work that you might want it to perform.

Below are the outputs of a reasoning- and code-related set of questions that also test the tool calling capabilities of the models. Here, the model returns the tool call but doesn’t execute the code itself.

```
→ uv run ollama_hard_reasoning_bench.py --model qwen3.6:35b
PASS debug_empty_tokenizer_regression: ok
PASS review_shell_command_injection: ok
FAIL choose_minimal_edit_for_cross_platform_path: argument instructions missing required content
FAIL triage_import_error_after_refactor: wrong tool: expected read_file, got ask_clarification
PASS debug_mutable_default_cache_leak: ok

Score: 3/5 passed (60.0%)
```

```
→ uv run ollama_hard_reasoning_bench.py --model north-mini-code-1.0
FAIL debug_empty_tokenizer_regression: wrong tool: expected final_answer, got edit_file
PASS review_shell_command_injection: ok
FAIL choose_minimal_edit_for_cross_platform_path: invalid JSON: Extra data: line 2 column 1 (char 235)
FAIL triage_import_error_after_refactor: wrong tool: expected read_file, got ask_clarification
FAIL debug_mutable_default_cache_leak: wrong tool: expected final_answer, got edit_file

Score: 1/5 passed (20.0%)
```

```

uv run ollama_hard_reasoning_bench.py --model gemma4:e2b
FAIL debug_empty_tokenizer_regression: wrong tool: expected final_answer, got edit_file
FAIL review_shell_command_injection: wrong tool: expected final_answer, got ask_clarification
FAIL choose_minimal_edit_for_cross_platform_path: wrong argument path: expected 'code/tool-
reasoning-benchmark/ollama_tool_reasoning_bench.py', got 'code/tool-reasoning-
benchmark/personal_tool_reasoning_tasks.jsonl'
FAIL triage_import_error_after_refactor: wrong tool: expected read_file, got ask_clarification
FAIL debug_mutable_default_cache_leak: wrong tool: expected final_answer, got edit_file

Score: 0/5 passed (0.0%)

```

For instance, we can say that `qwen3.6:35b` gets the conceptual debugging and security review tasks right, but still struggles with agentic judgment around “what file/action first” tasks. `3/5` is usable but not fully reliable for autonomous tool use. But a harness that constrains actions, adds retries, and maybe gives stronger project context could make it pretty usable.

On the other hand, `gemma4:e2b` failing `0/5` is a strong signal that it is less suitable for this of tool-use reasoning, even if it is fast. Note that the failures are not just formatting issues: looks like it chooses the wrong tool, asks for clarification when enough context is present. I would probably not use it as a coding-agent model beyond very narrow or heavily constrained tasks.

6. Agent Code Base Audit

Now, after this lengthy preamble setting up a local LLM, let’s get back to the main topic, the coding agent harness. As mentioned at the beginning of this article, we will use the `qwen3.6:35b` code (<https://github.com/QwenLM/qwen-code>) harness, as Qwen models have been optimized for it.

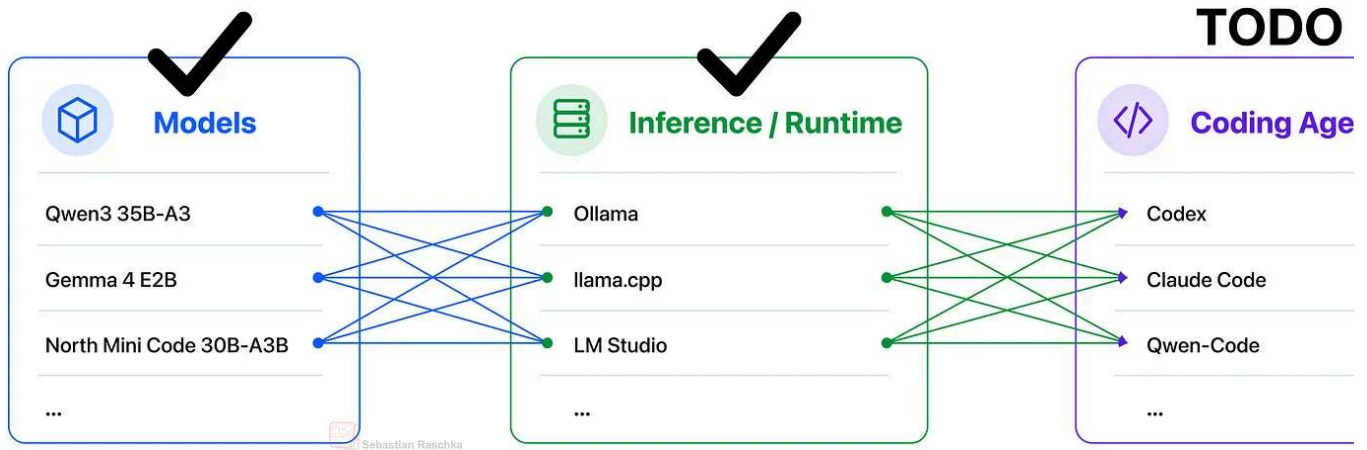


Figure 14: Next, we are trying to connect the locally served model to the coding agent harness.

If you are familiar with Claude Code, it's basically the same thing but fully open-source. However, I will also go over how to connect the local Qwen3.6 model to Codex and Claude Code in the next sections.

Note that coding harnesses are much more capable than LLMs by themselves. This is why I recommend being more careful about what you are running and where. For instance, when trying new (coding) agents, I like to

- Do an audit of the (open-source) agent code base first.
- Run it on separate hardware (e.g., my DGX Spark) or a separate user account and/or virtual environment on my machine at the very least.

Regarding the audit, I recommend looking for data sharing/egress and the default blast radius when it comes to file permissions, as well as some baseline robustness to prompt injection. The figure below attempts to summarize the main points.

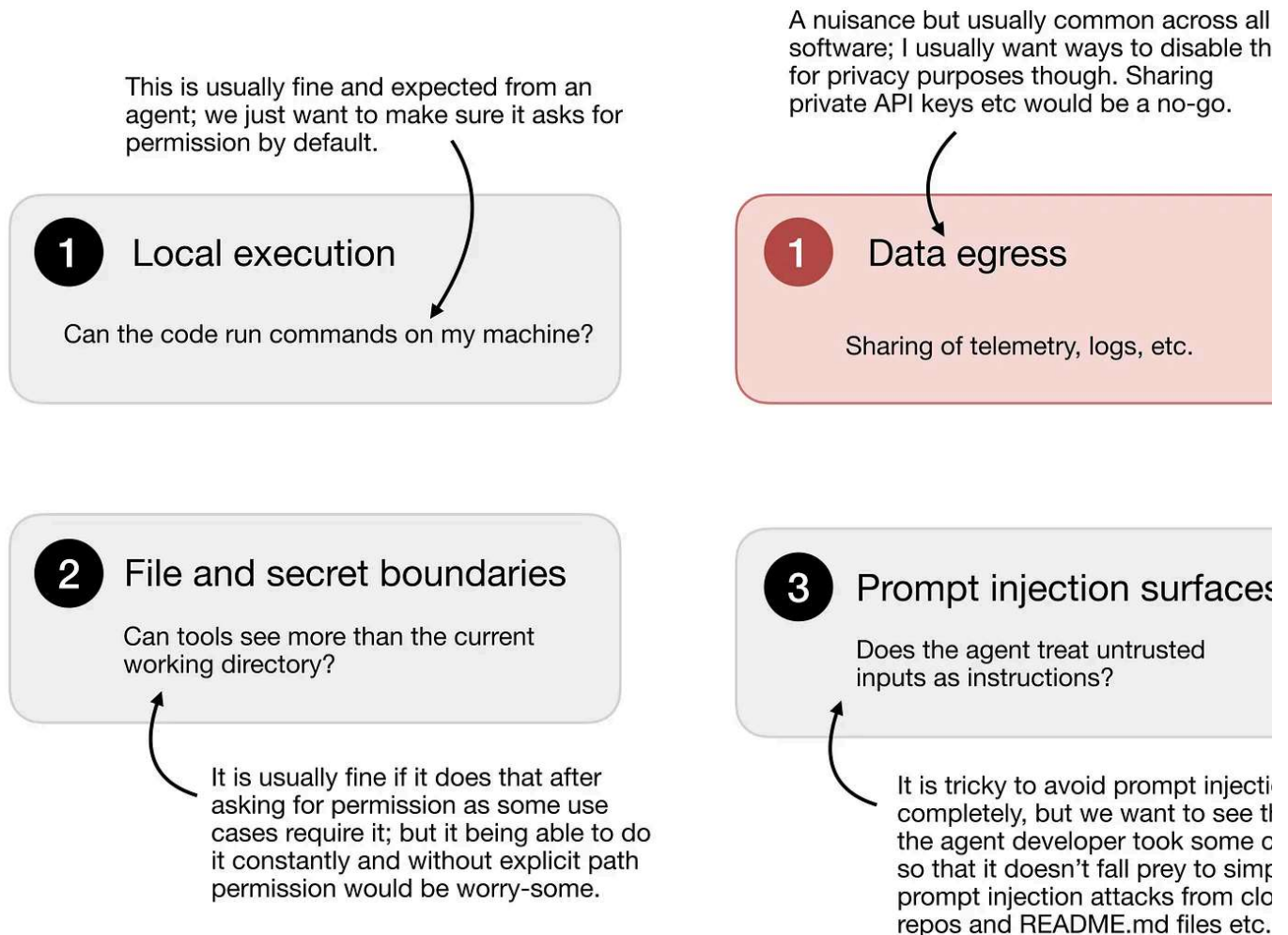


Figure 15: Practical audit checklist before running an installed coding agent harness.

Similar concerns apply to the local model serving engine (e.g., Ollama) as well. However, coding agents require even more attention as they can directly read data from your machine and manipulate files.

To do a basic audit, I recommend the following:

1. Clone the repo:

```
1 git clone https://github.com/QwenLM/qwen-code.git
```

2. Ask a trusted agent you used before (like GPT 5.5 in Codex or Opus 4.8 in Claude Code) to review it with a focused prompt. Something like the following:

You are auditing ./qwen-code before I install or run the agent on my machine.

Focus only on practical local-machine risk from the installed agent and the code path that create it:

- install scripts and package lifecycle hooks
- shell command execution by the agent
- file read/write boundaries at runtime
- secret handling and environment-variable inheritance
- how repo files, project instructions, and tool output can influence the agent
- MCP, plugin, extension, or tool integrations
- network calls and telemetry
- update mechanisms after installation
- terminal escape/output handling
- data egress and data residency

Ignoring internet downloads that are strictly required for installation, check whether the installed agent can send prompts, files, telemetry, logs, identifiers, or metadata to remote servers when I use a local model through Ollama. Ignore cloud-model configurations.

Do not infer risk from the project owner alone. Identify concrete endpoints, SDKs, default providers, environment variables, config defaults, and docs that control network behavior including any endpoints operated in foreign countries or by third-party companies.

Do not do broad style review. Do not refactor. Produce:

1. high-risk findings with file/line references
2. medium-risk concerns
3. network/data-egress findings, including any foreign, third-party, or China-linked endpoints or defaults
4. commands I should avoid running until reviewed
5. settings or environment variables that reduce local-machine risk
6. a short recommendation: safe to test in sandbox, safe to use, or do not run

For each item, say whether it is expected behavior for a coding agent or inherently riskier than Codex or Claude Code.

Below is a summary of the main findings (because the full report may be a bit boring and long for this article):

- 1. Local execution** Qwen Code can run shell commands on our machine through its tool but there are strict approval controls unless permissive modes such as `--yolo` are enabled. This is expected for a coding agent, and it's actually what makes it useful in practice. But of course it becomes risky if run unsandboxed or with a full environment containing secrets.
- 2. Data egress** Even with local Ollama, Qwen Code can send usage telemetry and metadata to Alibaba/Aliyun endpoints unless usage statistics and telemetry are disabled (more on that below). This is riskier than a local-only setup because model prompts stay local, but session IDs, tool metadata, model info, and local base URL metadata still leave the machine. But again, this is also common among all kinds of tools (yes, Codex and Claude do that as well).
- 3. File and secret boundaries** Workspace files are readable by default, while writes generally require approval and include some overwrite protections. This is good and standard agent practice.
- 4. Prompt injection surfaces** Repo instructions, tool output, MCP tools, extensions, and project config can influence the agent's behavior. Prompt injection attacks can be reduced via the approval gates mentioned above. This is normal for coding agents, but untrusted repos should be treated as hostile by default because they can steer the agent toward reading files, running commands, or sending data through approved tools.

Regarding the main privacy concerns in point 2, most of it is fixable via a custom `~/.qwen/settings.json` with the following contents:

```
1 {
2   "privacy": { "usageStatisticsEnabled": false },
3   "telemetry": { "enabled": false, "logPrompts": false },
4   "outboundCorrelation": { "propagateTraceContext": false },
5   "general": { "enableAutoUpdate": false },
6   "tools": {
7     "approvalMode": "default",
```

```
8     "sandbox": true
9   },
10  "mcpServers": {},
11  "hooks": { "disableAllHooks": true }
12 }
```

The `"general": { "enableAutoUpdate": false }` setting is a tradeoff. Security fixes not be installed automatically, but I prefer having explicit control over when updates happen instead of letting the tool pull and apply new code in the background.

By the way, cline (<https://github.com/Cline/Cline>), Codex (<https://github.com/openai/codex>) and Claude Code have similar telemetry data sharing defaults that would need to be disabled explicitly.

(Note that Claude Code doesn't have an official open-source version of their codebase, which makes trusting it even trickier, and it does seem to send data to both Anthropic and Datadog.)

Either way, overall, it seems Qwen-Code follows standard practices, and as of this writing there is no particular concern that is non-standard for coding agents.

7. Qwen-Code Setup

If we accept the reported findings and risks (personally, I didn't see any red flags), we can now proceed with the installation and hook up our local Qwen3.6-35B-A3B model to Qwen-Code (and Codex and Claude Code in the next sections).

As mentioned before, I preferably experiment with and run coding agents, which can read and edit local files, on a separate machine (in my case a DGX Spark, but it could also be a separate Mac or Linux workstation). Alternatively, I would run it in a VM or set up a separate macOS or Linux user account as a practical middle ground.

(I heard from some friends that they also rent servers for that, like Linode or Heroku, for tinkering purposes. However, instead of the monthly hosting costs for a somewhat capable machine, I would probably rather get a relatively cheap \$200-500 hardware box, or even an old retired laptop, and run a local harness and then use a stronger open-weight model hosted locally.)

in the cloud via Ollama cloud models, OpenRouter, etc if you are looking for alternatives to GPT or Claude.)

Anyways, let's install Qwen-Code. The listed options include, e.g.,

```
curl -fsSL https://qwen-code-assets.oss-cn-hangzhou.aliyuncs.com/installation/install-qwen-standalone.sh | bash
```

and

```
1  npm install -g @qwen-code/qwen-code@latest
```

However, running the commands above assumes that the published artifacts match the code we just reviewed in the GitHub repo. If we are extra careful/paranoid, we can also bootstrap ourselves from the GitHub repo. Be warned, this is more manual/messier though (I recommend executing them one at a time instead of copy & pasting the whole block into terminal):

```
1  # Go to your development folder
2  cd ~/Developer
3
4  # Clone the Qwen Code GitHub repository
5  git clone https://github.com/QwenLM/qwen-code.git
6
7  # Enter the cloned repository
8  cd qwen-code
9
10 # Install JavaScript dependencies
11 npm install
12
13 # Build the CLI output in the local dist/ folder
14 npm run build
15
16 # Create a user-level bin directory if it does not already exist
17 mkdir -p ~/.local/bin
18
19 # Create a qwen wrapper that runs the CLI from this source checkout.
20 # Keep ~/Developer/qwen-code in place, since this wrapper points into it.
21 cat > ~/.local/bin/qwen <<'SH'
22 #!/usr/bin/env sh
23 exec "$HOME/Developer/qwen-code/scripts/cli-entry.js" "$@"
```



```

24  SH
25
26  # Make the wrapper executable.
27  chmod +x ~/.local/bin/qwen
28
29  # Make qwen available in the current shell session.
30  export PATH="$HOME/.local/bin:$PATH"
31
32  # Verify that the qwen command is found and prints a version.
33  qwen --version

```

After completing the installation, we can now launch the Qwen-Code client via the `qwer` command from the terminal to complete the setup and connect to the locally served LLM.

For this, after running the `qwen` command, we select “Custom Provider”, as shown below

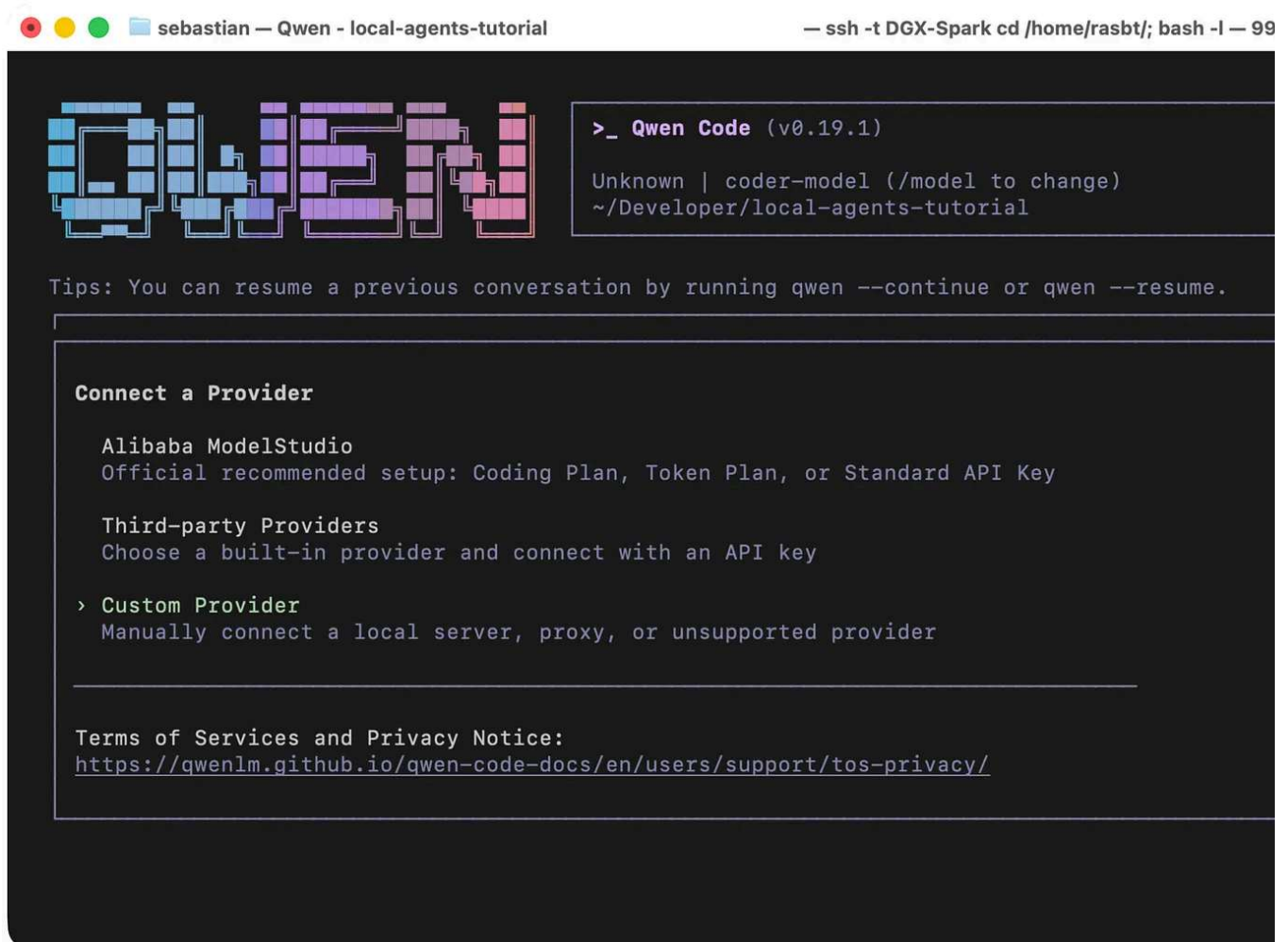


Figure 16: Choose “Custom Provider,” which lets us connect the Ollama LLM.

Ollama uses the OpenAI API standard. So, next, we follow the on-screen setup guide and choose the “OpenAI-compatible” option.

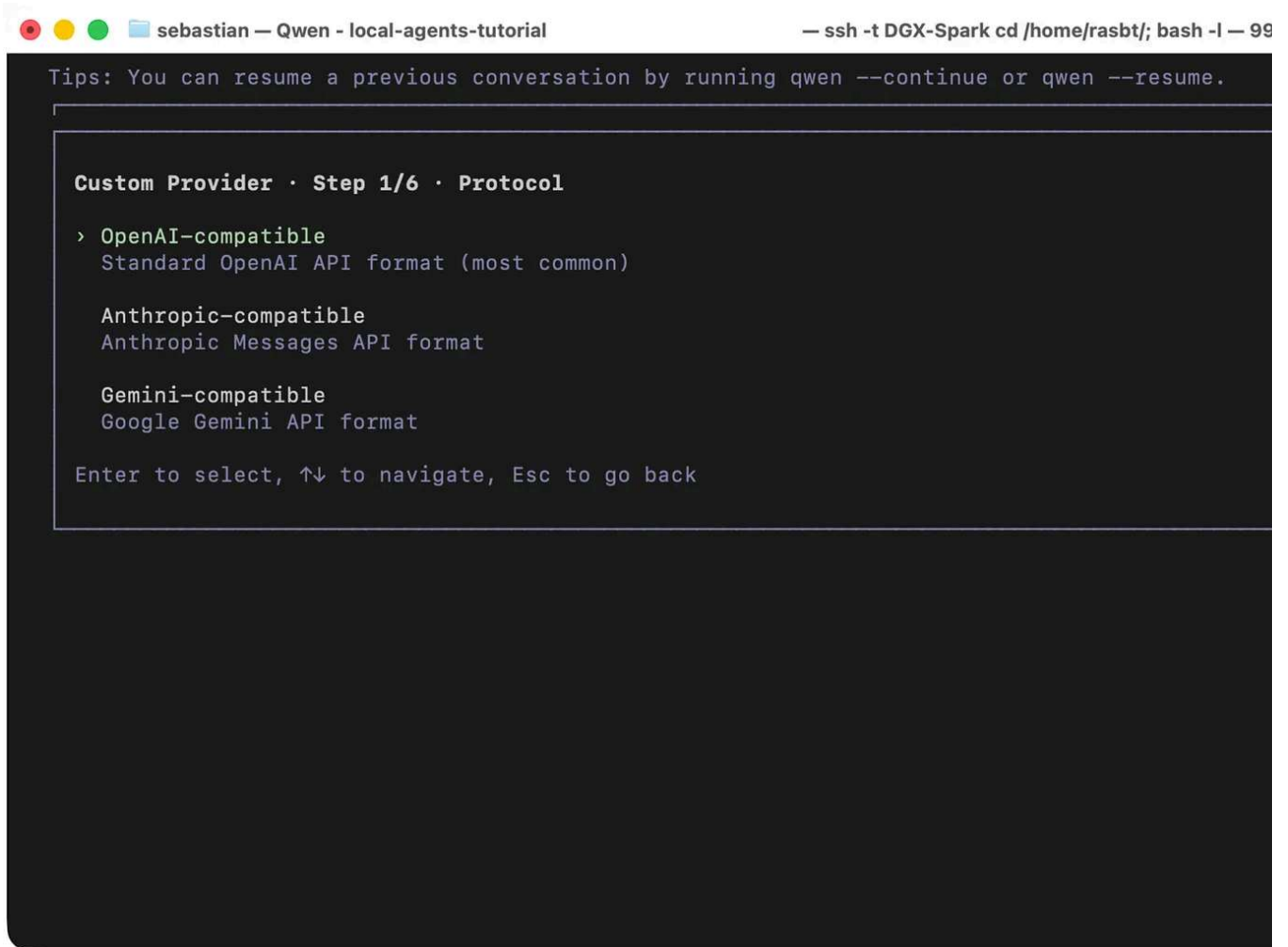
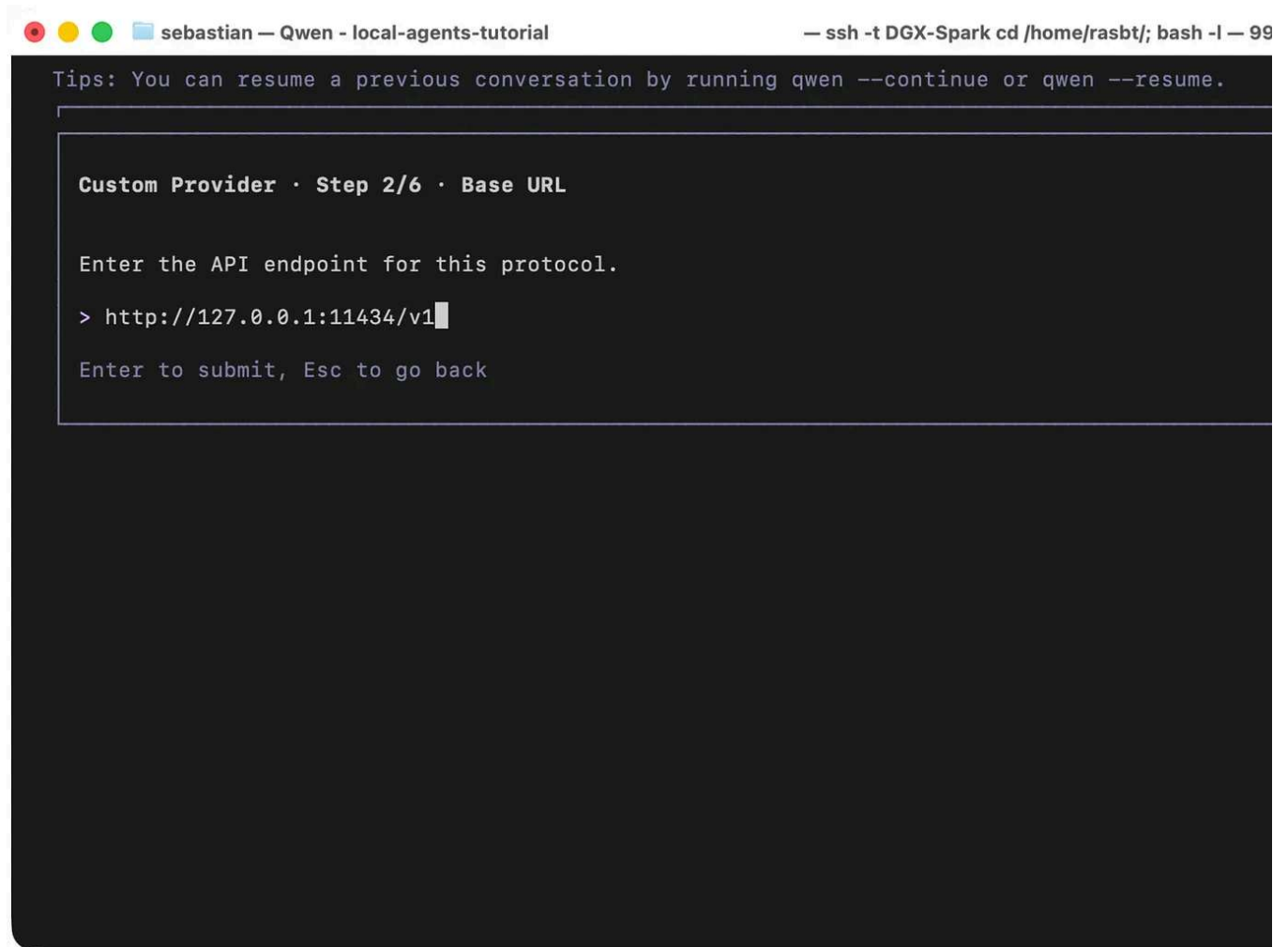


Figure 17: Since Ollama follows the OpenAI API standard, we choose “OpenAI-compatible” here.

Next, we need to provide the API endpoint of the running Ollama application that serves local LLM. Usually that’s the local

```
1 http://127.0.0.1:11434
```

address by default. We enter `http://127.0.0.1:11434/v1` (including the `/v1`) since that OpenAI-compatible base URL.



```
sebastian — Qwen - local-agents-tutorial — ssh -t DGX-Spark cd /home/rasbt/; bash -l — 99

Tips: You can resume a previous conversation by running qwen --continue or qwen --resume.

Custom Provider · Step 2/6 · Base URL

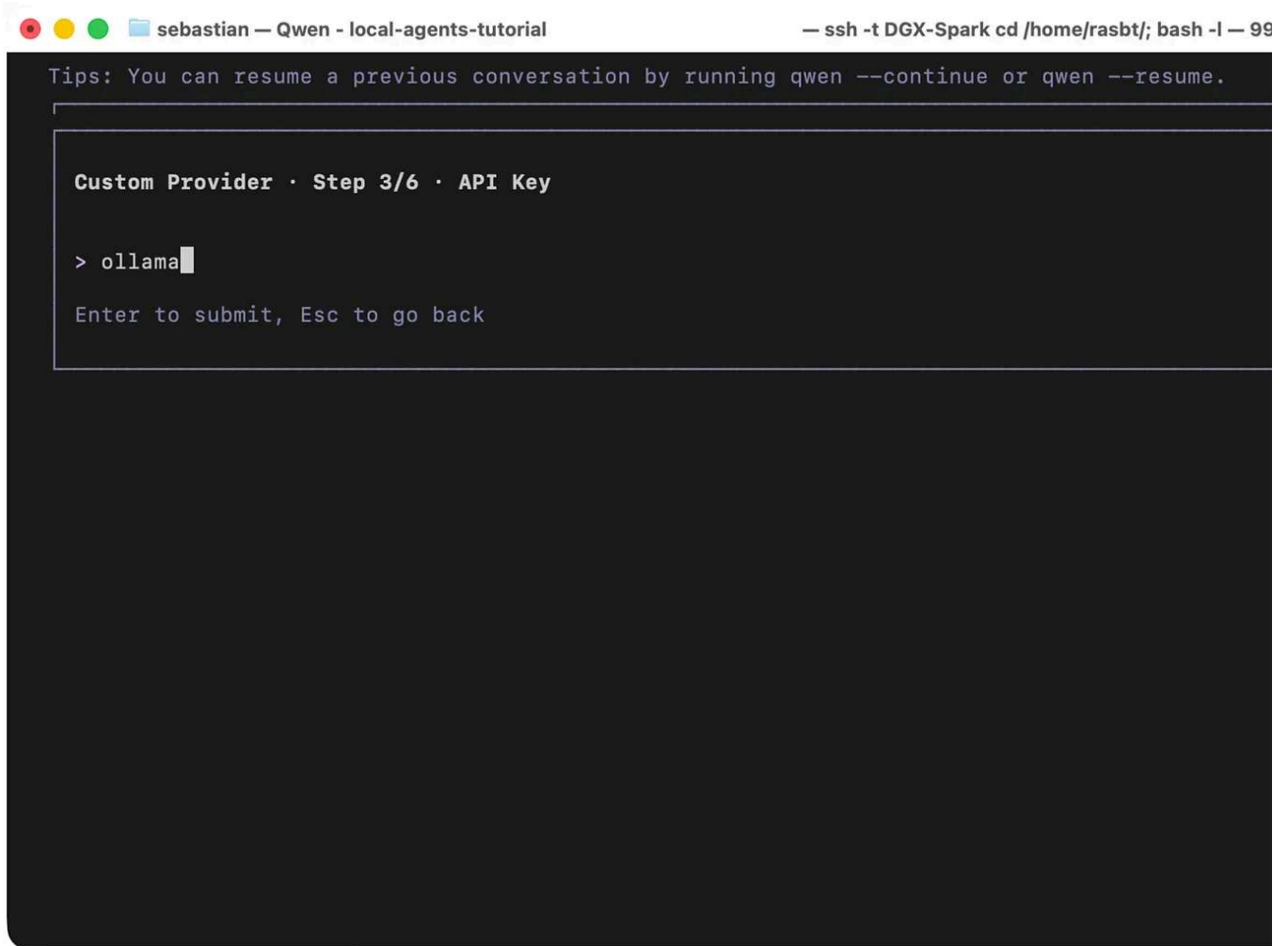
Enter the API endpoint for this protocol.

> http://127.0.0.1:11434/v1

Enter to submit, Esc to go back
```

Figure 18: Configure Qwen Code to use Ollama's local OpenAI-compatible endpoint, `http://127.0.0.1:11434/v1`.

Next, we enter `ollama` as our custom provider.



```
sebastian — Qwen - local-agents-tutorial — ssh -t DGX-Spark cd /home/rasbt/; bash -l — 99

Tips: You can resume a previous conversation by running qwen --continue or qwen --resume.

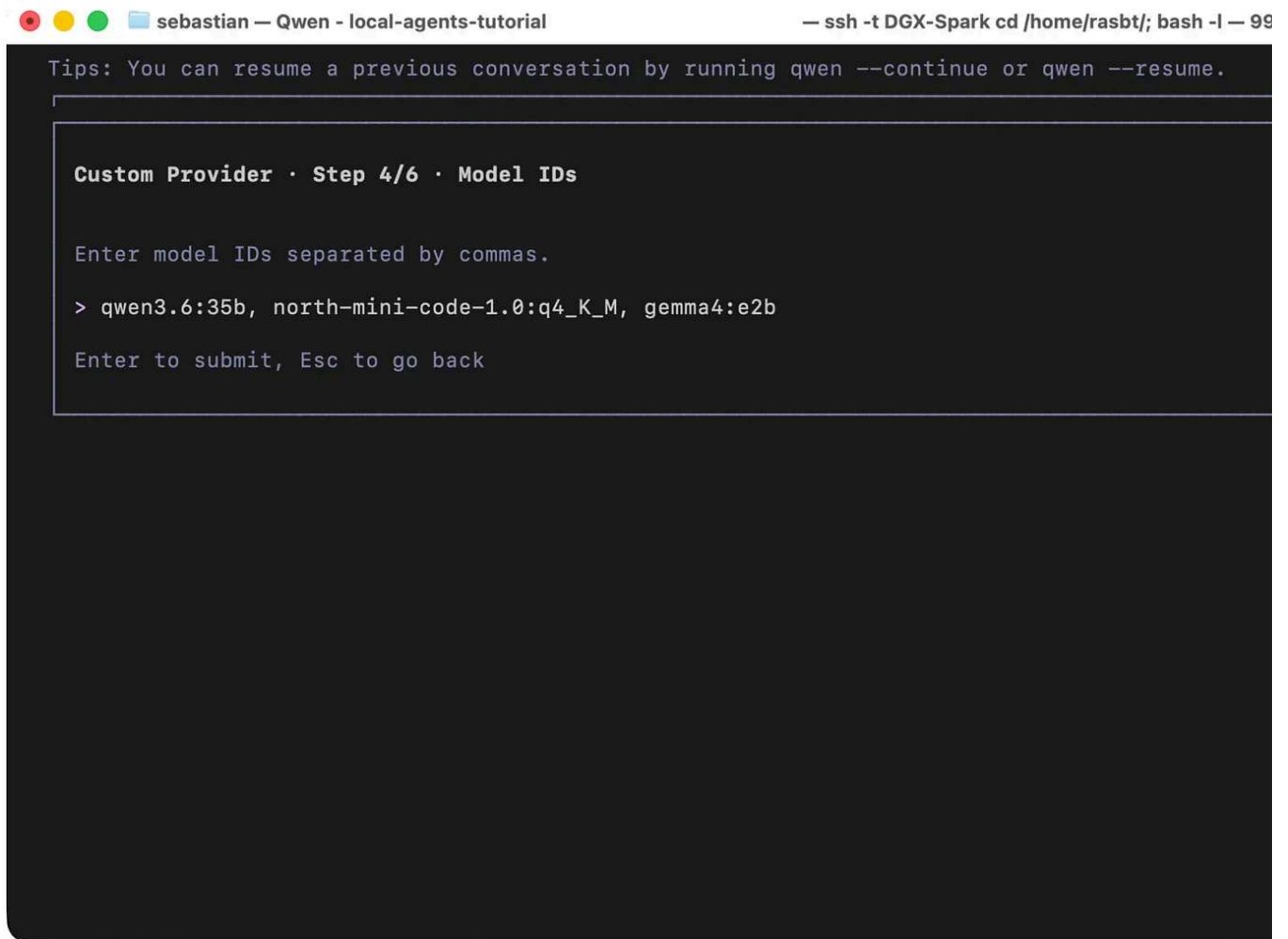
Custom Provider · Step 3/6 · API Key

> ollama

Enter to submit, Esc to go back
```

Figure 19: Enter `ollama` as the API key placeholder for the local custom provider.

Next, we can select the available models. These are the ones that we downloaded via `ollama pull`. You can enter only a single model or multiple ones separated by commas. You can double-check the list of downloaded models via `ollama list`. By the way, you can always add more models easily later (I'll explain after completing the setup).



```
sebastian — Qwen - local-agents-tutorial — ssh -t DGX-Spark cd /home/rasbt/; bash -l — 99

Tips: You can resume a previous conversation by running qwen --continue or qwen --resume.

Custom Provider · Step 4/6 · Model IDs

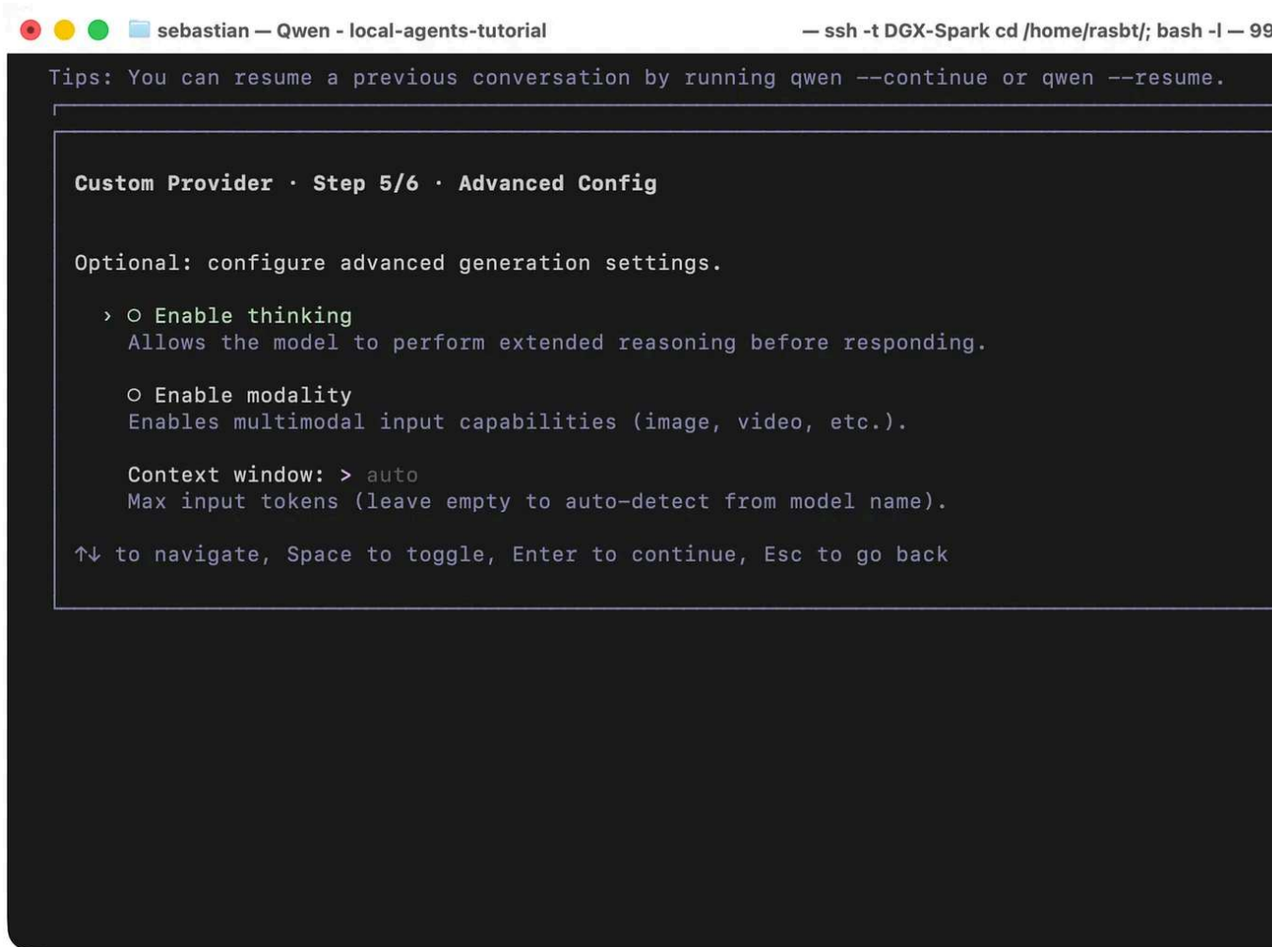
Enter model IDs separated by commas.

> qwen3.6:35b, north-mini-code-1.0:q4_K_M, gemma4:e2b

Enter to submit, Esc to go back
```

Figure 20: Select the local Ollama models that Qwen Code should make available through the custom provider.

We are almost done! In step 5/6, we of course select “Enable thinking” mode, which will result in higher token usage but the better resulting problem-solving capabilities are wor



```
sebastian — Qwen - local-agents-tutorial — ssh -t DGX-Spark cd /home/rasbt/; bash -l — 99

Tips: You can resume a previous conversation by running qwen --continue or qwen --resume.

Custom Provider · Step 5/6 · Advanced Config

Optional: configure advanced generation settings.

> ○ Enable thinking
  Allows the model to perform extended reasoning before responding.

  ○ Enable modality
  Enables multimodal input capabilities (image, video, etc.).

  Context window: > auto
  Max input tokens (leave empty to auto-detect from model name).

↑↓ to navigate, Space to toggle, Enter to continue, Esc to go back
```

Figure 21: Enable thinking mode for the local model provider.

And that's basically it. Step 6 is basically a review step that we can confirm by pressing "Enter".

Congratulations, you should now have a working fully-local LLM workflow set up. The usage is pretty much similar to Claude Code, where you can use `/` commands for various functionality. E.g., you can switch models via the `/model` command, as shown below.


```
sebastian — Qwen - local-agents-tutorial — ssh -t DGX-Spark cd /home/rasbt/; bash -l — 99

Tips: You can run any shell commands from Qwen Code using ! (e.g. !ls).
● Successfully configured Custom Provider. Use /model to switch models.

> /model

[
[
  Select Model

  > 1. [openai] qwen3.6:35b
    2. [openai] north-mini-code-1.0:q4_K_M
    3. [openai] gemma4:e2b

  Modality:      text-only
  Context Window: 131,072 tokens
  Base URL:      http://127.0.0.1:11434/v1
  API Key:       QWEN_CUSTOM_API_KEY_OPENAI_HTTP_127_0_0_1_11434_V1_A5A03F67A771

  Enter to select, ↑↓ to navigate, Esc to close
```

Figure 22: Use `/model` to switch models.

By the way, as I mentioned before, it's relatively easy to add new models from ollama. On you pull a new model via `ollama pull`, you can add it as a new entry in `~/qwen/settings.json`. Here, just copy & paste an existing entry into the file and change the "id" and "name" to that of the Ollama model name.

```

EXPLORER
OPEN EDITOR... 1 unsaved
{} settings.json ~/.qwen
Editor (3x) H: DGX-SPARK

> .ipynb
> .jupyter
> .keras
> .local
> .nv
> .oh-my-zsh
> .ollama
> .qwen
  > ide
  > projects
  > skills
  > tmp
  ≡ .file-history-cleanup
  ≡ .subagent-cleanup-3dee0071ed...
  ↓ output-language.md
  {} settings.json
  {} tip_history.json
> .ssh
> .triton
> .venv
> .vscode-server
> Desktop
> Developer

{} settings.json
28 "env": {
29   "QWEN_CUSTOM_API_KEY_OPENAI_HTTP_127_0_0_1_11434_V1_A5A03F67": "Ollama"
30 },
31 "modelProviders": {
32   "openai": [
33     {
34       "id": "qwen3.6:35b",
35       "name": "qwen3.6:35b",
36       "baseUrl": "http://127.0.0.1:11434/v1",
37       "envKey": "QWEN_CUSTOM_API_KEY_OPENAI_HTTP_127_0_0_1_11434_V1_A5A03F67",
38     },
39     {
40       "id": "north-mini-code-1.0:q4_K_M",
41       "name": "north-mini-code-1.0:q4_K_M",
42       "baseUrl": "http://127.0.0.1:11434/v1",
43       "envKey": "QWEN_CUSTOM_API_KEY_OPENAI_HTTP_127_0_0_1_11434_V1_A5A03F67",
44     },
45     {
46       "id": "gemma4:e2b",
47       "name": "gemma4:e2b",
48       "baseUrl": "http://127.0.0.1:11434/v1",
49       "envKey": "QWEN_CUSTOM_API_KEY_OPENAI_HTTP_127_0_0_1_11434_V1_A5A03F67",
50     },
51     {
52       "id": "xxxxxx",
53       "name": "xxxxxx",
54       "baseUrl": "http://127.0.0.1:11434/v1",
55       "envKey": "QWEN_CUSTOM_API_KEY_OPENAI_HTTP_127_0_0_1_11434_V1_A5A03F67",
56     }
57   ]
58 },
59 "requestUrl": {

```

Figure 23: We can add new ollama models by editing the `~/.qwen/settings.json` config file. Here, "xxxxxx" is the name of the ollama model name, e.g., "nemotron-3-nano:30b".

By the way, to update the qwen-code tool once in a while, if we used the git clone & local build route, we can pull a recent GitHub snapshot and update it as follows:

```

1  # Go to the local Qwen Code source checkout
2  cd ~/Developer/qwen-code
3
4  # Fetch the latest changes from GitHub
5  git pull
6
7  # Install or update dependencies if package files changed
8  npm install
9
10 # Rebuild the local CLI
11 npm run build
12
13 # Verify the updated CLI
14 qwen --version

```

8. Agent Capability Assessment

Now that we have a fully working, local coding agent, the question is: how well does it perform, and is it actually good enough for my tasks? Of course, there are benchmarks for this, but in my opinion, nothing beats trying it for yourself on some of your workflow. In other words, this basically means using it for a day or two to decide whether it meets your bar.

I also recommend compiling a small set of tasks that reflect your common coding agent usage. And if you come upon a particularly challenging one when working on a given project, it may not be a bad idea to add it to this set to evaluate future models.

As an example of what I mean, I shared a relatively small, simple, and general set of tasks that can be used to test the agents here on GitHub: <https://github.com/rasbt/local-coding-agent-evals/tree/main/agent-problem-pack>. This is basically an extension of the tasks from the Local LLM Setup section.

The details on how to run these are in the GitHub README: <https://github.com/rasbt/local-coding-agent-evals/tree/main/agent-problem-pack#quick-start-running-benchmarks-manually>.

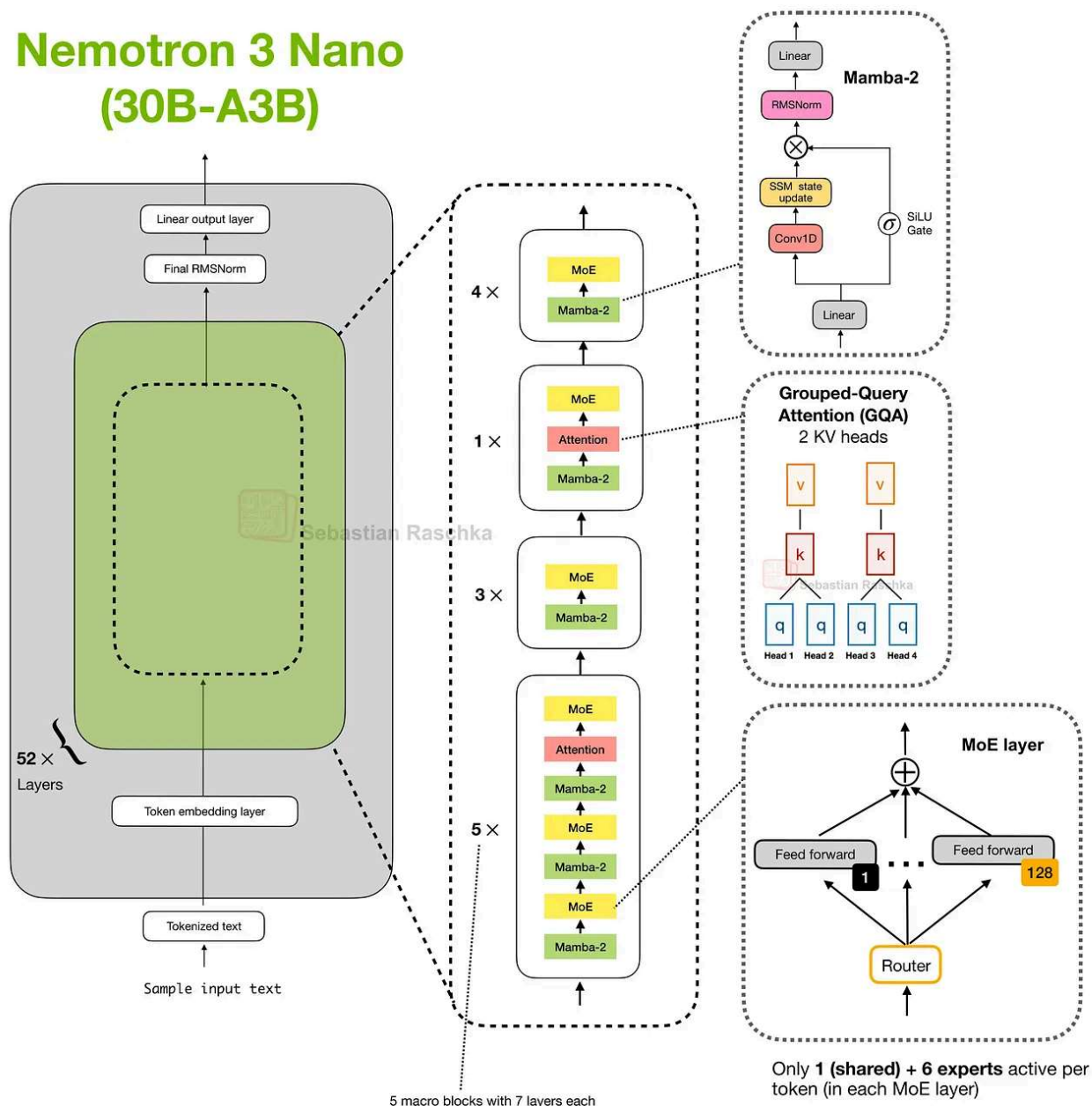
Below is the outcome for the different LLMs tested in Qwen-Code.

Model	problem-01-tokenizer-regression	problem-02-shell-command-injection	problem-03-cross-platform-task-path	problem-04-import-error-after-refactor	problem-05-mutable-default-cache	Overall
qwen3.6:35b	Pass	Pass	Fail	Pass	Pass, with concern	
north-mini-code-1.0:q4_K_M	Pass	Pass	Pass, suspicious	Fail	Pass	
gemma4:e2b	Pass	Fail	Fail	Fail	Fail	
nemotron-3-nano:latest	Pass	Pass, minor concern	Pass	Pass	Fail	

Figure 24: Small local agent capability benchmark using Qwen-Code. Code to reproduce:
<https://github.com/rasbt/local-coding-agent-evals>

As we can see, both the Qwen3.6 and North Mini Code 35B-A3B models solve 4 out of these problems. Gemma 4 E2B fails a lot. Out of curiosity, I also added the a bit older Nemotron 3 Nano model. It has a similar size and compute performance as the aforementioned Qwen and North models, and it performs similarly well.

Nemotron 3 Nano (30B-A3B)



DECODER TYPE Hybrid MoE	ATTENTION Mostly Mamba-2 with a few GQA layers	SCALE 30B total, 3B active (10% active)	CONTEXT (TOKENS) 1,000,000
LAYER MIX 6 GQA + 23 Mamba-2 + 23 MoE	KV CACHE / TOKEN (BF16) info 6 KiB · Very low	LICENSE NVIDIA Nemotron Open Model License	DATE 2025-12-04
KEY DETAIL Interleaves Mamba-2 and MoE blocks, using attention only sparingly.			

Figure 25: Nemotron 3 Nano architecture overview from my [LLM Gallery](#).

9. Codex Setup

After setting up the local coding agent (and the article exceeding 5000 words), this would probably be a reasonable place to stop. However, as a bonus, I also thought it might be interesting to add brief Codex and Claude Code notes for completeness.

Unfortunately, as far as I know, the Codex UI does not support non-OpenAI models, but you can use the Codex CLI to run our Ollama models.

If you haven't installed the OpenAI Codex CLI yet, you can get and install it analogously to qwen-code from their open-source GitHub directory: <https://github.com/openai/codex> (the Codex CLI is open source!)

I will spare you the lengthy listing of the commands and recommend checking the repo's README instead for the official instructions. (Cloning the repo and running an audit similar to qwen-code is not a bad idea here, as well.)

Then, once installed, there are multiple ways to enable local model use. In my opinion, the most convenient way is to set up a separate config `~/.codex/ollama.config.toml` (inside the existing `~/.codex` folder) with some default options:

```
1  model = "qwen3.6:35b"
2  model_provider = "ollama"
3  model_reasoning_effort = "high"
4  personality = "pragmatic"
5
6  [projects."/home/rasbt"]
7  trust_level = "trusted"
```

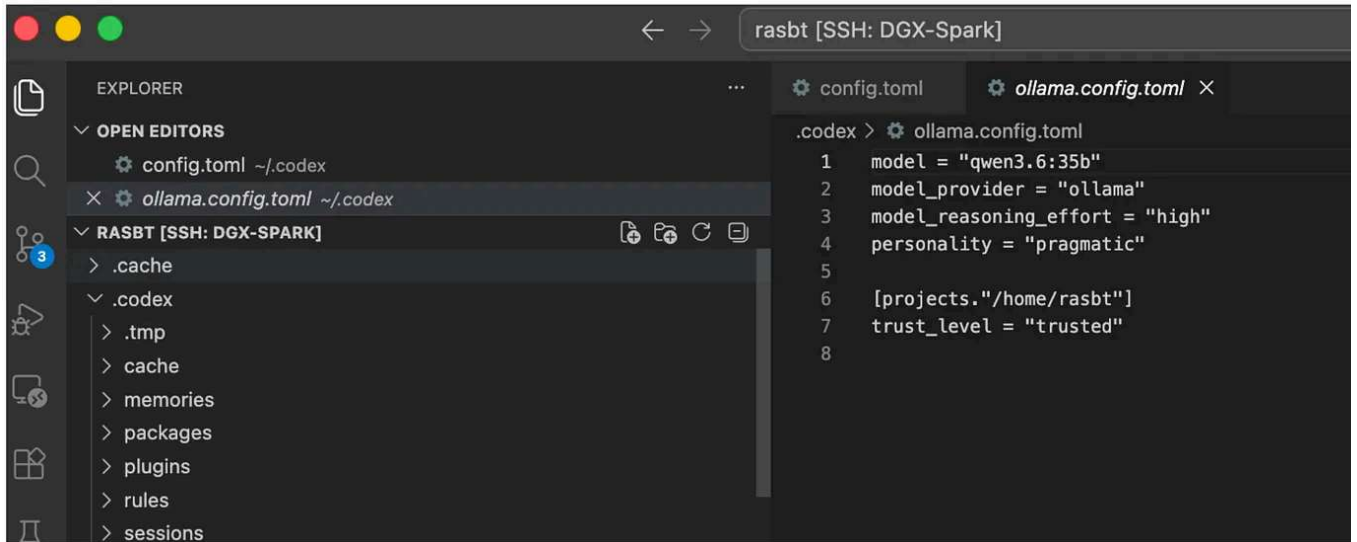



Figure 26: Set up a separate Ollama profile for Codex for convenience.

Then, we can still use `codex` to launch the regular “Codex with GPT 5.5” mode and use the Ollama model via `codex --profile ollama`.

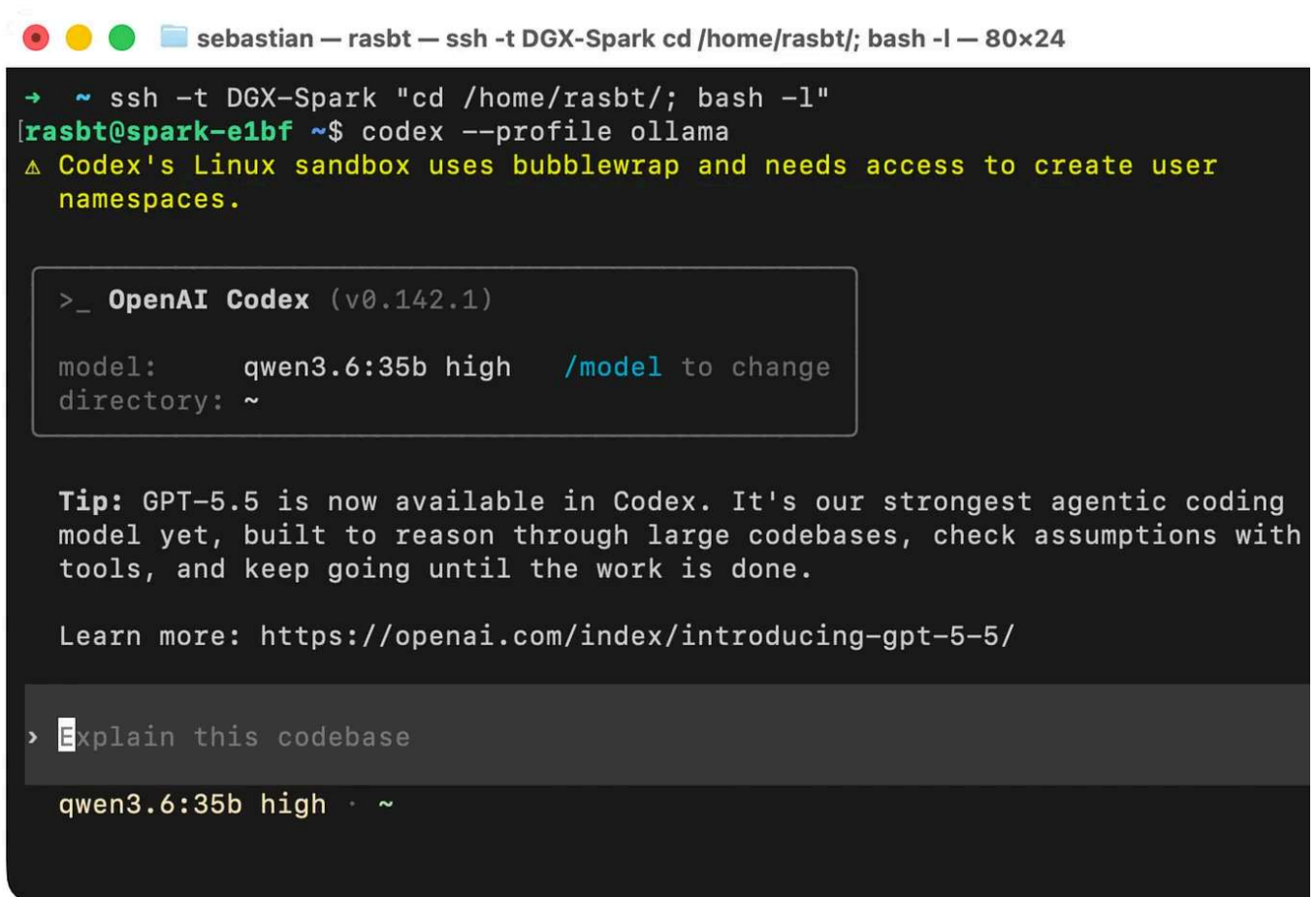


Figure 27: Launch Codex using a local Ollama model.

When rerunning the test cases from the Agent Capability Assessment section, to my surprise, Qwen3.6 does actually perform better via Codex compared to its “native” Qwen Code coding harness, as shown below.

Model	problem-01-tokenizer-regression	problem-02-shell-command-injection	problem-03-cross-platform-task-path	problem-04-import-error-after-refactor	problem-05-mutable-default-cache	Overall
qwen3.6:35b	Pass	Pass	Pass	Pass	Pass	
north-mini-code-1.0:q4_K_M	Pass	Pass	Pass	Pass	Pass	
gemma4:e2b	Fail	Fail	Fail	Fail	Fail	
nemotron-3-nano:latest	Pass	Pass	Pass	Pass	Pass	

Figure 28: Small local agent capability benchmark in Codex.

Even though this is just a small set of benchmarks, it suggests that using Codex as the universal coding agent harness may not be such a bad idea after all.

10. Claude Code Setup

Of course, there is also the popular Claude Code agent harness that we could use as a harness around our local LLMs. While very popular and capable, this is probably my least favorite option for local setups because the codebase is proprietary. That also means we cannot readily inspect and/or disable Anthropic’s data logging practices.

To set it up, if you don’t have Claude Code already installed on your machine, I suggest checking the official docs for recommended installation commands:

<https://code.claude.com/docs/en/quickstart>.

Claude Code itself does not expose the same local-provider configuration path as Code. However, Ollama provides an integration via `ollama launch claude`:

<https://docs.ollama.com/integrations/claude-code>

I.e., we can execute `ollama launch claude` to run the Claude Code harness with an Ollama model.

By the way, this also works for codex via `ollama launch codex`, but I personally prefer the `ollama launch codex --profile ollama` route we discussed earlier, as it gives me a bit more insight and control about how things work etc.

```

[rasbt@spark-e1bf ~]$ ollama launch claude
Claude Code v2.1.191

Welcome back!

qwen3.6 · API Usage Billing
/home/rasbt

Tips for getting started
Run /init to create a CLAUDE.md file with
Note: You have launched claude in your home directory

What's new
Added `/rewind` support for resuming a code edit
Fixed scroll position jumping to the bottom
Fixed background agents resurrecting after a crash
/release-notes for more

> |
? for shortcuts · ← for agents

```

Figure 29: Claude Code with a local Qwen3.6 model through Ollama.

However, as a user, it feels like Claude Code takes much longer to come up with a solution than Codex. This is probably because Claude Code has a much higher token usage. So, below, I additionally looked at the token usage of all three harnesses.

As we can see, Claude Code uses by far the most tokens on average, Codex the least.

Claude Code uses the most tokens overall

Total tokens across the same five local-agent tasks; lower is fewer tokens.

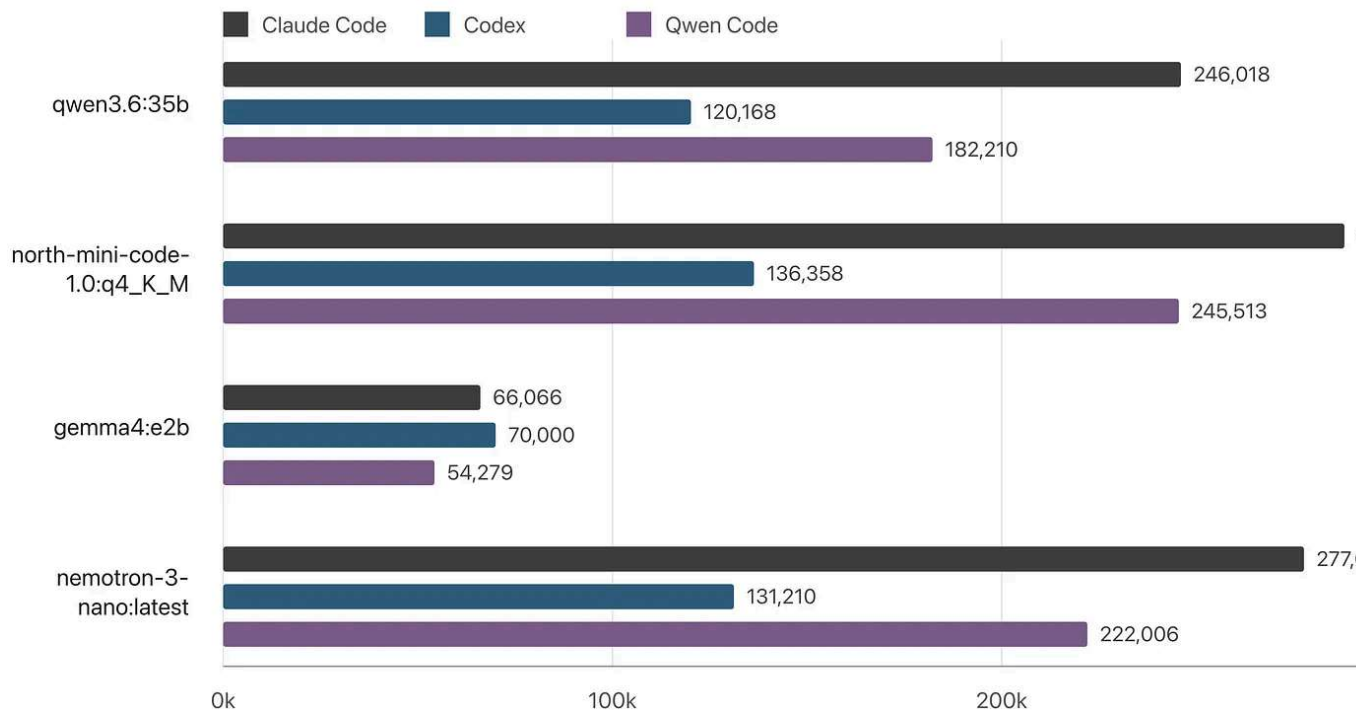


Figure 30: Average token usage of the three harnesses for different LLMs. Code to reproduce: <https://github.com/rasbt/local-coding-agent-evals>

When it comes to the little agent capability assessment benchmark, the Qwen and North Mini Code models also get 5/5, and even the small Gemma 4 model does ok!

Interestingly, we can also see that the token usage is largely driven by the harness, not the LLM itself. I.e., among all three LLMs that are capable of solving (almost) all 5 tasks, they use the same number of tokens (e.g., Qwen3.6 uses roughly the same number of tokens as North Mini Code and Nemotron 3 Nano when used inside Claude Code). Only Gemma 4 uses fewer tokens, but it also fails almost all tasks, likely because of insufficient tool-calling capabilities where the tasks interrupt early.

For reference, below is again the summarized task-success rate.

Harness	qwen3.6:35b	north-mini-code-1.0:q4_K_M	gemma4:e2b	nemotron-3-nano:lates
claude	5/5	5/5	3/5	4/5
codex	5/5	5/5	0/5	5/5
qwen-code	4/5	4/5	1/5	4/5

Figure 31: Summarized task success rates.

Anyway, the takeaway here is that if more tokens help the model-harness combination to solve more (and more complex) problems, great! But if we have two harnesses that both have an equal task success rate, a harness that uses 50% fewer tokens (e.g., Codex over Claude Code), then this is a huge win, because it will make tasks run twice as fast.

However, the big caveat here is that task correctness is a necessary criterion, but it does not measure code quality and readability, which are hard to assess automatically.

PS: I tried to analyze why Claude Code uses more tokens, and it seems that the difference mainly comes from input tokens rather than output tokens. In other words, Claude is not writing twice as much. The logs suggest that Claude is repeatedly feeding more context into the model across turns, including previous messages, tool calls, command outputs, and file contents. For example, one Claude run used about 578k input tokens but only about 100k output tokens across 25 turns. So the likely explanation is that Claude's harness accumulates or accounts for a larger prompt-side history during multi-step agent runs.

11. Mac <-> DGX

So far, all the setups we discussed assumed that we were running the local LLM on the same machine as the coding harness.

However, what if we developed some trust in the coding agent harness and want to use our main Mac while the model itself is hosted on a different machine, e.g., a DGX Spark?

In my opinion, the best (or most convenient) setup is an SSH tunnel from the Mac to the DGX.

First, I suggest quitting Ollama on the Mac or changing the 11434 to something else before.

Assuming we quit the Ollama app on the Mac, check that the following returns an empty output to indicate that Ollama is not available:

```
1 curl http://127.0.0.1:11434/v1/models
```

Then run the following command on that Mac in a terminal window on the Mac side:

```
1 ssh -N -L 11434:127.0.0.1:11434 rasbt@DGX-Spark
```

That command means that we open an SSH connection to DGX-Spark as user `rasbt`, which you need to adjust to whatever your username and machine name are. Then, the command forwards the Mac's local port 11434 to 127.0.0.1:11434 on the DGX because of `-L 11434:127.0.0.1:11434`. Note that this is the Ollama address.

The terminal running `ssh -N -L ...` will look like it is hanging. That is normal. Keep it open while you use Qwen Code, Codex, or Claude Code. Press `Ctrl-C` to stop the tunnel.

So after it is running, use this on your Mac to see if the Mac can indeed access the Ollama models from the DGX:

```
1 curl http://127.0.0.1:11434/v1/models
```

If that returns the DGX models, your Mac tools can use the DGX Ollama server as if it were local.

Then, just use Qwen Code and Codex just like above.

For Claude via `ollama launch claude`, the key is that the Mac-side `ollama` command must see the tunneled endpoint. If needed:

```
1 OLLAMA_HOST=http://127.0.0.1:11434 \  
2 ollama launch claude --model qwen3.6:35b
```


12. What about OpenClaw and Hermes?

We focused on Qwen Code, Codex, and Claude Code because they are the most direct coding-agent workflows. OpenClaw and Hermes are also capable, but they are broader agent harnesses. They are better suited when you want one agent to coordinate across 1 apps, browsers, terminals, and longer-running workflows.

For coding work, I recommend starting with Qwen Code, Codex, or Claude Code first (although there are also many other interesting coding harnesses like OpenCode, Cline, Pi, and Noumena Code). And I would treat OpenClaw and Hermes as interesting follow-up options for things beyond coding rather than the first baseline for this local coding-agent setup.

13. Conclusion

This was a long article with lots of information and configuration. If there are a few main takeaways, I'd say that it's not the mechanistic setup pipeline but rather the considerations when running coding agents locally. That is, the most important part is not getting one specific tool installed, but understanding the model-serving layer, the agent harness, the permission model, and how to evaluate whether the setup actually solves coding tasks reliably.

Of course, GPT 5.5 and Opus 4.8 are currently better than smaller open-weight models run on a Mac or DGX Spark. But the newer Mixture-of-Experts models in the 30-35B range (such as Qwen3.6, North Mini Code, and Nemotron 3 Nano) are all very, very capable and are really sufficient for a lot of tasks. And yes, they run with the same token speed as GPT 5.5 through a Pro subscription, so it should not necessarily slow down your workflows.

The main consideration when setting up local agents, besides the model itself, is also what harness we want to use. The common perception is that models are usually optimized for a specific harness than others (e.g., Qwen3.6 may work better in Qwen Code than Claude Code, for example). Based on the small agent assessment, this may not necessarily be true, though (this is only a very small benchmark, so take it with a big grain of salt). So, if you are more comfortable with a different harness that you have a lot of muscle memory with, like

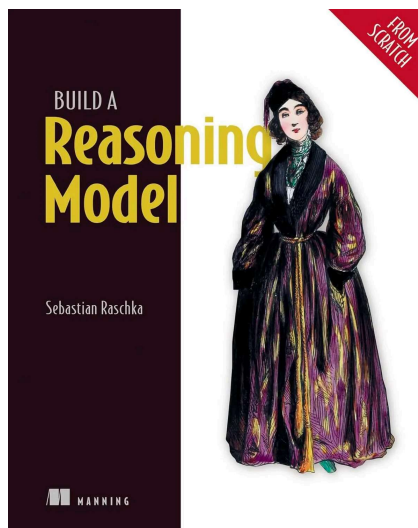
Codex and Claude Code, maybe it's not a bad idea to just stick the model into that one and give it a try!

Anyways, I hope the article was useful, and it got you interested in doing some tinkering with open-weight models. They are becoming more capable by the day, and it's for some inexplicable reason just fun to run models locally.

Further Resources

If you want to try the benchmarks yourself, the code and small evaluation tasks used in this article are available here: <https://github.com/rasbt/local-coding-agent-evals>

Also, my [Build a Reasoning Model \(From Scratch\)](#) book has now gone to print and starts shipping. I wanted to post a picture, but it will be 3 more days until it arrives.



[Build a Reasoning Model \(From Scratch\)](#).

If you liked my previous [Build a Large Language Model \(From Scratch\)](#) book, this is essentially a sequel implementing inference-time scaling techniques and reinforcement learning algorithms from scratch.

And if you want to support future long-form articles like this one, consider [becoming a paid subscriber](#). It helps me keep writing these independent deep dives and sharing the accompanying code, figures, and experiments.

Subscribe to Ahead of AI

By Sebastian Raschka · Thousands of paid subscribers

Ahead of AI focuses on machine learning and AI research and is read by more than 190,000 researchers and practitioners who want to stay ahead in a rapidly evolving field.

Upgrade to paid

By subscribing, you agree Substack's [Terms of Use](#), and acknowledge its [Information Collection Notice](#) and [Privacy Policy](#).



202 Likes · 24 Restacks

Discussion about this post

Comments Restacks



Write a comment...



Thomas R. Holy 2d

♥ Liked by Sebastian Raschka, PhD

Nice article, thank you very much! One thing i'am missing is a greater view on security. personally would put the coding instance and project files in a virtual machine and rem connect to it. So agents are clearly disconnected from private data and vulnerabilities are sandboxed to some temporary/check pointed system. However keeping your coding packages up to date requires agent web access features that can bring the risk of prompt injection. Do you have experience with safety guards for this, like building a extraction guard that omits such injections?

♥ LIKE (2) 💬 REPLY

3 replies by Sebastian Raschka, PhD and others



richardstevenhack 2d

♥ Liked by Sebastian Raschka, PhD

Terrific article!

I use MSTY Studio which is a GUI front-end to an embedded Ollama server. Check it out <https://msty.ai/> It's basically an alternative to things like LMStudio and AnythingLLM, but its own version of OpenClaw and can also support LLaMa.cpp and has an Agent Mode. Supports local and online models.

♥ LIKE (1) 💬 REPLY

2 replies by Sebastian Raschka, PhD and others

18 more comments...

© 2026 Raschka AI Research (RAIR) Lab LLC · [Privacy](#) · [Terms](#) · [Collection notice](#)
[Substack](#) is the home for great culture